ASSIGNMENT 1

CO1: *Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation*

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)

Nature of Assignment: Individual **Duration:** 1 Week
Submission Date: _____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K4 – Analyse

Analyse the parking problem and identify the objects that a Java program must model.

Discuss:

- The inputs the system receives and the outputs it must produce
- The nouns in the problem that become candidate classes, and the verbs that become methods
- The responsibilities of each candidate class
- The business rules every valid operation must satisfy (for example: a slot cannot hold two vehicles; a ticket number is never reused)
- Any assumptions you make about the facility

You are expected to **justify why each candidate class is needed**, not merely list names.

Alternative Concept and Complexity Analysis

In your design each vehicle is an object. If the vehicles were instead stored as plain Strings in parallel arrays (regNo[], type[], entryTime[]), what would the solution look like? Compare both designs for finding a vehicle by registration number and for adding a new vehicle attribute. Analyse the time complexity of the search and the space used per vehicle in each design.

Evidence Expected

A clear problem analysis with candidate classes, their responsibilities, business rules and stated assumptions.

Solution:

TASK 01 – Understand the Problem and Identify the Objects

1. Problem Analysis

The Campus Smart-Parking System is designed to manage vehicle parking in a multi-level university parking facility. It allows students, staff and visitors to park their vehicles in suitable available slots.

The system supports three types of vehicles: two-wheelers, cars and electric vehicles (EVs). When a vehicle enters, the system checks for an available slot, allocates the slot, generates a unique parking ticket and records the entry time.

When a vehicle leaves, the system releases the parking slot and calculates the parking fee based on vehicle type and parking duration. Students and staff with valid monthly passes can park free of charge, while visitors must pay the applicable fee.

2. Inputs and Outputs

Inputs	Outputs
Vehicle registration number	Unique parking ticket
Vehicle type	Allocated floor and slot
Entry time	Recorded entry details
Exit time	Parking duration
User category	Calculated parking fee
Monthly pass validity	Updated slot availability

3. Identification of Candidate Classes

The nouns in the problem statement are identified as candidate classes, while verbs represent the methods or operations.

Candidate class	Responsibilities
Vehicle	Stores registration number and vehicle details.
TwoWheeler	Represents two-wheeler vehicles and their parking rate.
Car	Represents cars and their parking rate.
ElectricCar	Represents electric vehicles and charging facilities.
ParkingSlot	Stores slot number and availability status.
ParkingFloor	Manages the parking slots on a particular floor.
ParkingFacility	Manages floors, allocates slots and releases slots.
ParkingTicket	Stores ticket number, vehicle details, entry time and exit time.
User	Stores user details and category.
ParkingPass	Maintains monthly pass validity for students and staff.

Justification:

Each class represents a separate real-world entity or responsibility. For example, Vehicle stores common vehicle information, while ParkingFacility manages the overall parking operation. Separating these responsibilities makes the system easier to maintain and extend.

4. Nouns and Verbs Identification

Nouns (Candidate Classes):

- Vehicle
- Car
- TwoWheeler

- ElectricCar
- ParkingSlot
- ParkingFloor
- ParkingFacility
- ParkingTicket
- User
- ParkingPass

Verbs (Methods):

Verbs	**Corresponding methods**
Enter	vehicleEntry()
Allocate	allocateSlot()
Generate	generateTicket()
Record	recordEntryTime()
Release	releaseSlot()
Calculate	calculateFee()
Validate	validatePass()
Exit	vehicleExit()

5. Business Rules

The following rules must be satisfied by the parking system:

1. A parking slot can accommodate only one vehicle at a time.
2. A vehicle must be allocated only to an available and suitable slot.
3. Every parking ticket must have a unique ticket number.
4. A vehicle cannot enter again with the same registration number while it is already parked.
5. The parking slot must be released when the vehicle leaves.
6. Parking fees depend on vehicle type and parking duration.
7. Students and staff with valid monthly passes are exempted from parking fees.
8. Visitors must pay the applicable parking fee.
9. Electric vehicles must use designated charging slots when charging is required.
10. Invalid vehicle details and invalid parking slot numbers must be rejected.

6. Assumptions

1. Every vehicle has a unique registration number.

2. The parking facility has multiple floors with a fixed number of slots on each floor.
3. Each vehicle occupies only one parking slot.
4. Parking charges are calculated based on completed or started hours according to the pricing rules.
5. A valid monthly pass is checked during fee calculation.
6. All entry and exit times are recorded by the system.
7. The system uses simulated parking data for implementation and testing.

7. Alternative Concept and Complexity Analysis

Alternative: Using Parallel Arrays Instead of Objects

In the object-oriented approach, each vehicle is represented as a separate Vehicle object containing its registration number, type and entry time.

Instead, the system could store vehicle details using three parallel arrays:

```
String[] regNo = new String[100];

String[] type = new String[100];

String[] entryTime = new String[100];
```

All three arrays use the same index to represent one vehicle.

For example:

Index	**regNo[]**	**type[]**	**entryTime[]**
0	TN01AB1111	Car	09:00
1	TN02CD2222	TwoWheeler	09:30
2	TN03EF3333	ElectricCar	10:00

Comparison of Both Approaches

Criteria	**Vehicle Objects**	**Parallel Arrays**
Data storage	Each vehicle is an object	Data stored in separate arrays
Searching	Linear search, O(n)	Linear search, O(n)
Adding a new attribute	Add a field to Vehicle	Create another parallel array
Data management	Easier to manage	Requires matching array indexes
Maintainability	Better	More difficult for large systems

Time Complexity:

- Object-oriented approach: Finding a vehicle by registration number requires checking each vehicle object. Time complexity is O(n).
- Parallel array approach: The registration number array must be searched sequentially. Time complexity is O(n).

Space Complexity:

- Object-oriented approach: O(n) for storing n vehicle objects.
- Parallel array approach: O(n) for storing three arrays containing n entries each.

The object-oriented approach has additional object and reference overhead, while parallel arrays require separate storage for each attribute.

Conclusion:

The object-oriented approach is preferred because it combines related vehicle information into a single object. Adding a new attribute is easier, and the system becomes more readable, reusable and maintainable. Therefore, the object-oriented design is more suitable for the Campus Smart-Parking System.

TASK 2 – Choose the Data Representation 3 Marks

Bloom's Level: K4 – Analyse

Decide how the data of the system will be represented using Java data types and arrays.

Discuss:

- The data type chosen for each field (slot number, vehicle number, entry time, duration, fee) and why
- How the floors and slots will be represented (one-dimensional, two-dimensional or jagged array) and why
- Where type conversion or casting is required (for example: converting minutes to hours for billing) and the risk of losing precision
- How you will read input (Scanner or command-line arguments)

Show at least one example where **integer division or an incorrect cast would produce a wrong fee**, and how your design avoids it.

Alternative Concept and Complexity Analysis

Instead of a jagged array String[floor][slot], suppose the slots are stored in a single onedimensional array using index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. What would change in your solution when floors have different numbers of slots? Analyse the time to locate a slot and the space required in each representation.

Evidence Expected

Justified choice of data types and array structure, with an identified casting risk and its resolution.
TASK 02 – Choose the Data Representation

1. Selection of Java Data Types

Field	Data type	Reason
Floor number	int	Stores floor index
Slot number	int	Stores slot index
Vehicle number	String	Stores registration number
Entry time	LocalDateTime	Stores date and time
Exit time	LocalDateTime	Stores exit date and time
Duration	long	Stores parking duration in minutes
Parking fee	double	Stores decimal fee values
Vehicle type	String or enum	Identifies vehicle category
Pass validity	boolean	Checks whether the pass is valid

2. Representation of Floors and Slots

I choose a jagged two-dimensional array to represent the parking facility.

```
String[][] slots = {
   new String[20],
   new String[50],
   new String[100]
};
```

- Each row represents one parking floor.
- Each column represents a parking slot.
- null indicates an available slot.
- A vehicle registration number indicates an occupied slot.

Reason: A jagged array allows different floors to have different numbers of parking slots.

3. Type Conversion and Casting

Type conversion is required when calculating parking duration and fees.

For example:

- Minutes are converted into hours.
- Integer values are converted into decimal values during fee calculation.

Incorrect calculation:

int minutes = 90;

double hours = minutes / 60;

Output: 1.0

This happens because integer division removes the decimal part.

Correct calculation:

int minutes = 90;

double hours = minutes / 60.0;

Output: 1.5

Using 60.0 ensures floating-point division and avoids precision loss from integer division.

4. Reading Input

I choose the Scanner class to read user input because it is simple and suitable for a console-based Java application.

Scanner sc = new Scanner(System.in);

System.out.print("Enter vehicle number: ");

String regNo = sc.nextLine();

System.out.print("Enter vehicle type: ");

String type = sc.nextLine();

5. Alternative Concept and Complexity Analysis

Alternative 1: One-Dimensional Array

String[] slots = new String[150];

The index is calculated using:

index = floor × slotsPerFloor + slot

This works easily when every floor has an equal number of slots. When floors have different slot counts, the calculation requires maintaining the starting index of each floor.

Alternative 2: ArrayList

ArrayList<String[]> floors = new ArrayList<>();

Each element stores one floor's slot array. It supports adding floors dynamically.

6. Comparison

Representation	**Slot access**	**Space complexity**

Jagged array	O(1)	O(S)
1D array	O(1)	O(F × S)
ArrayList of arrays	O(1)	O(S)

Here, F represents the number of floors and S represents the total number of parking slots.

Conclusion

I choose a jagged two-dimensional array because different floors may contain different numbers of parking slots. It provides direct slot access, efficient memory usage and flexibility when the parking facility is expanded.

TASK 3 – Design the Control Logic 4 Marks

Bloom's Level: K4 – Analyse

Design the decision and looping logic for the two core operations: **allocating a free slot** and **computing the parking fee**.

Discuss:

- How the program searches the floors and slots for a suitable free slot, and when the search stops
- How the fee slabs (for example: first hour, each additional hour, daily maximum) are expressed using if–else or switch
- How break, continue and return are used to avoid unnecessary work
- What happens when no suitable slot is free

Write **pseudocode for both operations** following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing Java code.

Complete the Code

Complete the class below and execute it. Do not change the main() method.

```
public class ParkingFacility {
   // slots[floor][slot] holds a vehicle number, or null if the slot is free     private final String[][] slots;

   public ParkingFacility(int[] slotsPerFloor) {         slots = new
String[slotsPerFloor.length][];         for (int f = 0; f < slotsPerFloor.length; f++) {
slots[f] = new String[slotsPerFloor[f]];
      }
   }

   /** Returns {floor, slot} of the first free slot, or null if it is full. */     public int[] allocate(String vehicleNo) {
      // TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first
      //   free slot and return {floor, slot}


      return null;     }

   /** Frees a slot. Returns false if the position is invalid or already free. */     public boolean release(int floor, int slot) {
      // TODO 2: Validate floor and slot, free the slot and return true; otherwise
      //   return false


   }

   public static void main(String[] args) {
      // floor 0 has 2 slots, floor 1 has 1 slot
      ParkingFacility p = new ParkingFacility(new int[]{2, 1});
      System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
      System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
            + p.release(5, 0));
      System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
   }
}
```

Expected output:

```
[0, 0]
[0, 1] [1, 0]
null
true false false
[0, 1]
```

Alternative Concept and Complexity Analysis

Your allocation scans every floor and slot until a free one is found. If, instead, you maintained a count of free slots per floor and a stack of free slot numbers, what would the allocation and release logic become? Analyse the time complexity of allocate() and release() and the extra space needed in both approaches for F floors and S slots per floor.

Evidence Expected

Complete pseudocode for slot allocation and fee computation with correct decision and loop logic.

Solution:

TASK 03 – Design the Control Logic

1. Slot Allocation Logic

Slot allocation is the process of finding an available parking slot for an incoming vehicle.

The system searches each floor and checks every slot. If a free slot is found, the vehicle number is stored in that slot and its floor and slot positions are returned.

If all slots are occupied, the system returns null.

Steps involved:

1. Start searching from the first floor.
2. Check each slot on that floor.
3. If a slot is free, allocate it to the vehicle.
4. Return the floor and slot number.
5. If no free slot is available, return null.

2. Parking Fee Calculation Logic

The parking fee depends on the vehicle type and parking duration.

For example, the following pricing rules are assumed:

Parking duration	**Fee rule**
First hour	Base hourly rate
Every additional started hour	Additional hourly rate
Daily maximum	Maximum ₹200 per day

The system calculates the billable hours and applies the appropriate fee.

For example, if a car's hourly rate is ₹30 and it parks for 130 minutes:

- Billable hours = 3
- Parking fee = 3 × 30
- Total fee = ₹90

4. Use of Control Statements

Statement	**Purpose**
if-else	Checks whether a slot is free and applies fee rules.
for loop	Searches floors and slots.
break	Stops searching after finding a free slot.
continue	Skips occupied slots.
return	Returns the allocated slot or calculated fee.

5. Pseudocode – Slot Allocation

```
ALGORITHM AllocateSlot(vehicleNo)
BEGIN
   FOR each floor f
      FOR each slot s in floor f
         IF slot[f][s] is occupied THEN
            CONTINUE
         END IF
         slot[f][s] = vehicleNo
         RETURN f, s
      END FOR
   END FOR
   RETURN NULL
END
```

6. Pseudocode – Parking Fee Calculation

Assumption: Every started hour is charged in full, with a daily maximum of ₹200.

```
ALGORITHM CalculateFee(minutes, ratePerHour)
BEGIN
   IF minutes <= 0 THEN
      RETURN 0
   END IF
   hours = minutes / 60
   IF minutes MOD 60 != 0 THEN
      hours = hours + 1
   END IF
   fee = hours * ratePerHour
   IF fee > 200 THEN
      fee = 200
```

END IF

RETURN fee

END

7. Alternative Concept and Complexity Analysis

Alternative: Using a Free-Slot Stack

Instead of searching every floor and slot, the system can maintain a stack of available slot positions for each floor.

- During allocation, take a free slot from the stack.
- During release, return the slot to the stack.
- Maintain a free-slot count for each floor.

Operation	**Scanning approach**	**Stack approach**
Allocate	O(F × S) worst case	O(1)
Release	O(1) with direct position validation	O(1)
Extra space	O(F × S) for parking data	O(F × S) for stacks in addition to parking data

Here, F is the number of floors and S is the maximum number of slots per floor.

The stack approach provides faster allocation but requires additional memory to maintain free-slot information.

8. Java Implementation – Complete the Given Code

Mandatory execution

What you have to do:

1. Open VS Code.
2. Open your Task03 folder.
3. Create a Java file named ParkingFacility.java.
4. Copy the complete code below.
5. Run the program.
6. Compare your output with the expected output in the question.
7. Take a screenshot and save it as output.png.

Complete Java program

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number,
    // or null if the slot is free
    private final String[][] slots;
    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }
    /** Returns {floor, slot} of the first free slot,
     * or null if it is full.
     */
    public int[] allocate(String vehicleNo) {
        // TODO 1 completed
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }

        return null;
    }
    /** Frees a slot. Returns false if the position
     * is invalid or already free.
```

```
 */
public boolean release(int floor, int slot) {
    // TODO 2 completed
    if (floor < 0 || floor >= slots.length) {
        return false;
    }
    if (slot < 0 || slot >= slots[floor].length) {
        return false;
    }
    if (slots[floor][slot] == null) {
        return false;
    }
    slots[floor][slot] = null;
    return true;
}
public static void main(String[] args) {
    ParkingFacility p = new ParkingFacility(new int[]{2, 1});
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN01AB1111")));
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN02CD2222")));
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN03EF3333")));
    System.out.println(java.util.Arrays.toString(
        p.allocate("TN04GH4444")));
    System.out.println(p.release(0, 1) + " "
        + p.release(0, 1) + " "
        + p.release(5, 0));
```

```
        System.out.println(java.util.Arrays.toString(
            p.allocate("TN04GH4444")));
    }
}
```

Expected Output

[0, 0]

[0, 1]

[1, 0]

null

true false false

[0, 1]

TASK 4 – Design Classes, Objects and Constructors 4 Marks

Bloom's Level: K4 – Analyse

Design the classes identified in Task 1 and decide how their objects will be created.

Discuss:

- The fields and methods of each class
- Default, parameterised and overloaded constructors, and when each is used
- Use of the this keyword and constructor chaining to avoid repeated code
- Whether a copy constructor is needed anywhere (for example: re-issuing a duplicate ticket) and why
- How objects are stored in memory (reference variables, stack and heap)

Alternative Concept and Complexity Analysis

You used overloaded constructors and constructor chaining. If objects were instead created with a default constructor followed by setter calls, or through static factory methods, what would the solution be? Compare how each approach guarantees a valid object, and analyse the time and space cost of creating n ParkingTicket objects in each approach.

Evidence Expected

Class definitions with justified constructors, use of this and constructor chaining, and an explanation of object creation.

Solution:

TASK 4 – Design Classes, Objects and Constructors

1. Introduction

The Campus Smart-Parking System uses object-oriented programming to represent parking facilities, vehicles, parking slots, and parking tickets. Each class contains suitable fields, methods, and constructors to manage parking operations.

2. Classes, Fields and Methods

Class	Fields	Methods
ParkingFacility	facilityName, floors	allocateSlot(), releaseSlot()
ParkingSlot	slotNumber, floorNumber, isOccupied	occupy(), release()
Vehicle	vehicleNumber, vehicleType	getVehicleDetails()
ParkingTicket	ticketId, vehicle, entryTime, exitTime, fee	calculateFee(), closeTicket()

3. Constructors

A constructor initializes an object when it is created.

A. Default constructor

A constructor without parameters.

```
class Vehicle {
    String vehicleNumber;
    String vehicleType;

    Vehicle() {
        vehicleNumber = "UNKNOWN";
        vehicleType = "CAR";
    }
}
```

Use: Creates an object with default values.

B. Parameterised constructor

A constructor that accepts parameters.

```
class Vehicle {
    String vehicleNumber;
```

```
    String vehicleType;

    Vehicle(String vehicleNumber, String vehicleType) {
        this.vehicleNumber = vehicleNumber;
        this.vehicleType = vehicleType;
    }
}
```

Use: Creates a vehicle with specific details.

C. Overloaded constructors

A class can have multiple constructors with different parameter lists.

```
class ParkingTicket {
    int ticketId;
    String vehicleNumber;

    ParkingTicket() {
        this(0, "UNKNOWN");
    }

    ParkingTicket(int ticketId) {
        this(ticketId, "UNKNOWN");
    }

    ParkingTicket(int ticketId, String vehicleNumber) {
        this.ticketId = ticketId;
        this.vehicleNumber = vehicleNumber;
    }
}
```

Use: Allows tickets to be created with different levels of information.

4. Use of this Keyword and Constructor Chaining

The this keyword refers to the current object. It helps distinguish instance variables from constructor parameters.

Example:

this.ticketId = ticketId;

Constructor chaining means calling one constructor from another using this().

Example:

```
ParkingTicket() {
   this(0, "UNKNOWN");
}
```

Benefits:

- Avoids repeated initialization code.
- Improves code reusability.
- Makes constructors easier to maintain.
- Ensures consistent object initialization.

5. Copy Constructor

A copy constructor creates a new object using the values of an existing object.

Example:

```
class ParkingTicket {
   int ticketId;
   String vehicleNumber;

   ParkingTicket(int ticketId, String vehicleNumber) {
      this.ticketId = ticketId;
      this.vehicleNumber = vehicleNumber;
   }

   ParkingTicket(ParkingTicket other) {
      this(other.ticketId, other.vehicleNumber);
```

```
    }
}
```

Use in the parking system:

A copy constructor can be useful when creating a duplicate ticket containing the same vehicle details. However, a re-issued ticket should normally receive a new unique ticket ID and a new issue time. Therefore, copying every field directly may not be appropriate.

6. Object Creation and Memory Allocation

In Java, objects are generally created using the new keyword.

Example:

```
ParkingTicket ticket1 = new ParkingTicket(101, "TN01AB1234");
```

Memory allocation:

- Stack: Stores local variables and reference variables such as ticket1.
- Heap: Stores the actual ParkingTicket object created using new.
- Reference variable: Holds the reference to the object in memory.

When an object is no longer reachable, Java's garbage collector can reclaim its memory.

Understanding Stack and Heap Memory in Java: Detailed Explanation with Examples and Use Cases | by Vinotech | Medium

medium.com

7. Alternative Concept and Complexity Analysis

Three approaches can be used to create ParkingTicket objects.

A. Default constructor followed by setter methods

```
ParkingTicket ticket = new ParkingTicket();

ticket.setTicketId(101);

ticket.setVehicleNumber("TN01AB1234");
```

- Easy to create and modify objects.
- May temporarily create an incomplete or invalid object.
- Validation must be performed in setter methods or before using the object.

B. Parameterised constructor with constructor chaining

```
ParkingTicket ticket =

    new ParkingTicket(101, "TN01AB1234");
```

- Initializes all required fields together.
- Helps ensure the object is valid at creation.
- Constructor chaining avoids repeated initialization code.

C. Static factory method

A static factory method creates and returns an object.

```
class ParkingTicket {
   int ticketId;
   String vehicleNumber;
   private ParkingTicket(int ticketId, String vehicleNumber) {
      this.ticketId = ticketId;
      this.vehicleNumber = vehicleNumber;
   }
   static ParkingTicket createTicket(int id, String number) {
      if (id <= 0 || number == null || number.isBlank()) {
         throw new IllegalArgumentException("Invalid ticket details");
      }
      return new ParkingTicket(id, number);
   }
}
```

Use:

```
ParkingTicket ticket =
   ParkingTicket.createTicket(101, "TN01AB1234");
```

- Provides better control over object creation.
- Can validate data before returning the object.
- Can return different object types or reuse existing objects when appropriate.

8. Time and Space Complexity

Let n be the number of ParkingTicket objects created.

Approach	Time complexity	Space complexity

Default constructor + setters	O(n)	O(n)
Parameterised constructor	O(n)	O(n)
Static factory method	O(n)	O(n)

Explanation:

- Creating one ticket takes constant time, O(1), assuming a fixed number of fields and simple validation.
- Creating n tickets takes O(n) time.
- Storing n ticket objects requires O(n) space.
- All three approaches have the same asymptotic complexity, although validation and method-call overhead may differ.

9. Conclusion

The Campus Smart-Parking System uses classes and objects to represent real-world parking entities. Parameterised and overloaded constructors help initialize objects with valid details. Constructor chaining and the this keyword reduce repeated code. Copy constructors can be useful for duplicating ticket information, while static factory methods provide additional control over object creation.

Recommended approach: Use parameterised constructors with constructor chaining for ParkingTicket objects because they provide clear initialization and help maintain valid object states.

TASK 5 – Apply static Members and the Object Life Cycle 3 Marks

Bloom's Level: K4 – Analyse

Identify the data and behaviour that belong to the class as a whole rather than to individual objects.

Discuss:

- Where static variables are needed (for example: a ticket counter, total occupancy) and why an instance variable would be wrong
- Where static methods or a static block are appropriate
- What happens to a ParkingTicket object after the vehicle leaves, and when it becomes eligible for garbage collection
- How objects are passed to and returned from methods in your design

Alternative Concept and Complexity Analysis

The ticket counter is a static variable. If it were made an instance variable of ParkingFacility, what would happen when the university runs two facilities (north and south gates)? Propose the solution for each case and analyse the space used by the counter for k facility objects and n tickets.

Evidence Expected

Correct and justified use of static members with an explanation of object passing and the object life cycle.

Solution:

TASK 5 – Apply Static Members and the Object Life Cycle (3 Marks)

Bloom's Level: K4 – Analyse

1. Introduction

In the Campus Smart-Parking System, static members are used to store data and perform operations that belong to the entire class rather than individual objects.

For example, a ticket counter and total parking occupancy can be maintained using static variables. Static members help share common information among all objects.

2. Static Variables

Static variables are declared using the static keyword. They are shared by all objects of a class.

Static variable	**Purpose**
ticketCounter	Generates unique ticket numbers.
totalOccupancy	Tracks the total number of occupied slots.
totalTicketsIssued	Counts the tickets issued by the system.

Example:

```
class ParkingTicket {

   static int ticketCounter = 1000;



   int ticketId;

   String vehicleNumber;



   ParkingTicket(String vehicleNumber) {

      ticketId = ++ticketCounter;

      this.vehicleNumber = vehicleNumber;

   }

}
```

Why is a static variable needed?

The ticket counter must be shared by all ParkingTicket objects. If it were an instance variable, every ticket object would maintain a separate counter, which could generate duplicate ticket numbers.

3. Static Methods and Static Blocks

A. Static Method

A static method belongs to the class and can be called without creating an object.

Example:

```
class ParkingFacility {
    static int totalOccupancy = 0;

    static void displayOccupancy() {
        System.out.println("Total Occupancy: " + totalOccupancy);
    }
}
```

Usage:

```
ParkingFacility.displayOccupancy();
```

B. Static Block

A static block is executed only once when the class is loaded. It is used to initialize static variables.

Example:

```
class ParkingTicket {
    static int ticketCounter;

    static {
        ticketCounter = 1000;
    }
}
```

Uses:

- Static methods display common parking information.
- Static blocks initialize shared values.
- They avoid unnecessary object creation for class-level operations.

4. Object Life Cycle of a ParkingTicket

A ParkingTicket object is created when a vehicle enters the parking facility.

The object stores:

- Unique ticket number.
- Vehicle registration number.
- Entry time.
- Exit time.
- Parking fee.

When the vehicle leaves, the ticket is closed, the parking slot is released and the fee is calculated.

After the ticket is no longer referenced by the program, the object becomes eligible for garbage collection.

Example:

```
ParkingTicket ticket = new ParkingTicket("TN01AB1234");


// Vehicle leaves

ticket = null;
```

Here, ticket = null removes the reference to the object. If no other references exist, the object becomes eligible for garbage collection.

Garbage Collection: Java automatically identifies and reclaims memory occupied by objects that are no longer reachable.

5. Passing and Returning Objects from Methods

In Java, objects can be passed as arguments to methods and returned from methods.

Example:

```
class ParkingService {


   void displayTicket(ParkingTicket ticket) {

      System.out.println(ticket.vehicleNumber);

   }


   ParkingTicket createTicket(String number) {

      return new ParkingTicket(number);
```

```
    }
}
```

Explanation:

- displayTicket() receives a ParkingTicket object as an argument.
- createTicket() creates and returns a ParkingTicket object.
- Object references allow different classes to work together.

6. Alternative Concept and Complexity Analysis

The ticket counter is maintained as a static variable in the original design.

Suppose the university has two parking facilities: North Gate and South Gate.

Alternative: Using an Instance Variable

If ticketCounter is an instance variable of ParkingFacility, each facility maintains its own counter.

Example:

```
class ParkingFacility {
    int ticketCounter = 1000;
}
ParkingFacility north = new ParkingFacility();
ParkingFacility south = new ParkingFacility();
```

Both facilities start with the same counter value. Therefore, duplicate ticket numbers may be generated.

Solution 1: Static Counter

```
class ParkingTicket {
    static int ticketCounter = 1000;
}
```

Both facilities share the same counter, so ticket numbers are generated from a common sequence.

Solution 2: Separate Counter for Each Facility

Each facility can maintain its own counter with a unique facility prefix.

For example:

- North Gate: N-1001
- South Gate: S-1001

This avoids duplicate ticket numbers between facilities.

Comparison

Criteria	Static counter	Instance counter
Storage	Shared by all objects	Separate for each facility
Counter values	Common sequence	Independent sequence
Duplicate risk	Lower with shared counter	Possible across facilities
Memory usage	One counter	One counter per facility

Time and Space Complexity

Let:

- k = Number of parking facility objects.
- n = Number of parking tickets.

Criteria	Static counter	Instance counter
Ticket generation	O(1)	O(1)
Creating n tickets	O(n)	O(n)
Counter space	O(1)	O(k)
Ticket storage	O(n)	O(n)

The static counter requires constant space because only one shared counter is maintained. An instance counter requires additional space for every facility object.

7. Conclusion

Static members are useful for maintaining common information such as ticket numbers and total occupancy in the Campus Smart-Parking System. Static methods perform class-level operations, while static blocks initialize shared data.

When a vehicle leaves, its ParkingTicket object becomes eligible for garbage collection once it is no longer reachable.

Recommended approach: Use a static ticket counter to generate unique ticket numbers across multiple parking facilities. This reduces memory usage and avoids duplicate ticket numbers within the shared system.

TASK 6 – Design the Inheritance Hierarchy 4 Marks

Bloom's Level: K4 – Analyse

Design an inheritance hierarchy for the vehicles and, if appropriate, for the users (student, staff, visitor).

Discuss:

- The superclass and subclasses, and the type of inheritance used (single, multilevel or hierarchical)
- What is inherited and what is added in each subclass
- How super is used for fields, methods and constructors
- The order in which constructors execute when an ElectricCar object is created, shown by a trace

Draw the hierarchy as a **UML class diagram**.

Alternative Concept and Complexity Analysis

You used inheritance (Car is-a Vehicle). If composition were used instead – a single Vehicle class that has-a VehicleType object holding the rate – what would the solution be? Compare the work needed to add a new vehicle type and analyse the time to compute a fee and the space per vehicle object in both designs.

Evidence Expected

A justified inheritance hierarchy, UML diagram, use of super and a traced constructor execution order.

Solution:

TASK 6 – Design the Inheritance Hierarchy

Campus Smart-Parking System – 4 Marks

1. Inheritance Hierarchy

Inheritance allows one class to acquire the fields and methods of another class. In the Campus Smart-Parking System, inheritance is used to represent different vehicle types and user categories.

Vehicle hierarchy

- Vehicle – Superclass containing common vehicle details.
- TwoWheeler – Subclass representing two-wheelers.
- Car – Subclass representing cars.
- ElectricCar – Subclass of Car representing electric cars.

The design uses both hierarchical and multilevel inheritance.

- Hierarchical inheritance: Vehicle → TwoWheeler and Vehicle → Car.
- Multilevel inheritance: Vehicle → Car → ElectricCar.

User hierarchy

- User – Superclass containing common user details.
- Student – Subclass representing student users.
- Staff – Subclass representing college staff.
- Visitor – Subclass representing visitors.

This uses hierarchical inheritance.

2. UML Class Diagram

Campus Smart-Parking System – Inheritance UML

Conceptual class hierarchy. For the written diagram, ElectricCar extends Car; Student, Staff and Visitor extend User.

3. Inherited and Added Members

Class	**Inherited members**	**New members**
Vehicle	–	vehicleNumber, ownerName, baseRate
TwoWheeler	Vehicle fields and methods	helmetRequired
Car	Vehicle fields and methods	numberOfSeats
ElectricCar	Vehicle and Car members	batteryCapacity
User	–	userId, name
Student	User members	rollNumber
Staff	User members	employeeId
Visitor	User members	visitPurpose

4. Use of the super Keyword

The super keyword refers to the immediate parent class.

It is used in three ways:

1. Access parent fields: super.vehicleNumber
2. Call parent methods: super.displayInfo()
3. Call parent constructor: super(vehicleNumber, ownerName, baseRate)

Example:

```
class Vehicle {
   String vehicleNumber;
   String ownerName;
   double baseRate;

   Vehicle(String vehicleNumber, String ownerName, double baseRate) {
      this.vehicleNumber = vehicleNumber;
```

```
        this.ownerName = ownerName;
        this.baseRate = baseRate;
        System.out.println("Vehicle constructor");
    }
    void displayInfo() {
        System.out.println("Vehicle: " + vehicleNumber);
    }
}
class Car extends Vehicle {
    int numberOfSeats;
    Car(String vehicleNumber, String ownerName,
        double baseRate, int numberOfSeats) {
        super(vehicleNumber, ownerName, baseRate);
        this.numberOfSeats = numberOfSeats;
        System.out.println("Car constructor");
    }
}

class ElectricCar extends Car {
    double batteryCapacity;

    ElectricCar(String vehicleNumber, String ownerName,
                double baseRate, int numberOfSeats,
                double batteryCapacity) {

        super(vehicleNumber, ownerName, baseRate, numberOfSeats);
        this.batteryCapacity = batteryCapacity;
        System.out.println("ElectricCar constructor");
```

```
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Battery Capacity: " + batteryCapacity + " kWh");
    }
}
```

5. Constructor Execution Trace

When an ElectricCar object is created:

```
ElectricCar e1 = new ElectricCar(
    "TN01EV1234", "Arun", 20.0, 5, 75.0
);
```

The constructor execution order is:

1. Vehicle constructor

Initializes vehicle number, owner name and base rate.

2. Car constructor

Initializes the number of seats.

3. ElectricCar constructor

Initializes battery capacity.

Output

Vehicle constructor

Car constructor

ElectricCar constructor

Java executes the superclass constructor before the subclass constructor.

6. Alternative Concept – Composition vs Inheritance

Composition means a class contains an object of another class instead of inheriting from it.

For example, a Vehicle class can have a VehicleType object that stores the vehicle category and parking rate.

Comparison	Inheritance	Composition
Design	Car extends Vehicle	Vehicle has a VehicleType
Adding a new type	Create a new subclass	Create a new VehicleType
Fee calculation	Uses subclass rate or overridden method	Uses VehicleType rate
Flexibility	Less flexible when types change	More flexible
Object space	Parent and subclass fields	Vehicle fields plus a type reference
Maintenance	May create many subclasses	Type-specific logic is separated

Time Complexity

- Inheritance: Fee calculation takes O(1) time when the subclass directly provides its rate.
- Composition: Fee calculation also takes O(1) time when the VehicleType object directly provides its rate.

Space Complexity

Let k be the number of vehicle types and n be the number of vehicles.

- Inheritance: Each vehicle object stores common fields and its subclass-specific fields. Total vehicle storage is O(n), with class definitions for k types.
- Composition: Each vehicle stores a reference to a VehicleType object. If type objects are shared, the additional type storage is O(k), making the overall storage O(n + k). If every vehicle creates its own type object, additional storage can be O(n).

Conclusion: Inheritance is suitable for the Campus Smart-Parking System because vehicles share common properties but have different characteristics. Composition is more flexible when new vehicle types and parking rates need to be added frequently.

TASK 7 – Override and Specialise Behaviour 4 Marks

Bloom's Level: K4 – Analyse

Identify the behaviour that must differ between subclasses and implement it by method overriding.

Discuss:

- Which methods are overridden (for example: calculateFee()) and why the subclass version is needed
- Use of the @Override annotation and the rules of overriding (signature, return type, access)
- Where the final keyword is appropriate for a variable, method or class, and why
- How toString(), equals() and hashCode() are overridden so that two Vehicle objects with the same registration number are treated as equal

Alternative Concept and Complexity Analysis

Each subclass overrides the fee method. If a single calculateFee() with a switch on a type field were used instead of overriding, what would the solution be? Analyse the time per fee calculation for k vehicle types and the number of classes that must change when a new type is added.

Evidence Expected

Correct overriding with @Override, justified use of final, and correct Object-class method overrides.

Solution:

TASK 7 – Override and Specialise Behaviour

Campus Smart-Parking System – 4 Marks

1. Method Overriding

Method overriding allows a subclass to provide its own implementation of a method inherited from its superclass.

In the Campus Smart-Parking System, different vehicles have different parking fees. Therefore, the calculateFee() method is overridden in each vehicle subclass.

- Vehicle – declares the common fee calculation method.
- TwoWheeler – calculates the fee using the two-wheeler rate.
- Car – calculates the fee using the car rate.
- ElectricCar – calculates the fee using the electric car rate.

2. Use of @Override Annotation

The @Override annotation indicates that a subclass method overrides a superclass method.

Rules of method overriding:

1. The method name and parameter list must match.
2. The return type must be the same or a compatible covariant return type.
3. The access level cannot be more restrictive than the parent method.
4. Static methods cannot be overridden; they are hidden.
5. A final method cannot be overridden.

Example:

```
@Override
public double calculateFee(int hours) {
   return hours * 10.0;
}
```

3. Use of the final Keyword

The final keyword prevents unwanted changes.

Usage	Example	Purpose
Final variable	final String registrationNumber	Prevents changing the registration number
Final method	final void displayRules()	Prevents subclasses from overriding the method
Final class	final class ParkingRules	Prevents inheritance

In this system, the vehicle registration number is declared final because it should not change after the vehicle is created.

The calculateFee() method should not be final because each subclass needs to override it.

4. Overriding Object Class Methods

Every Java class inherits methods from the Object class.

Three important methods are:

- toString(): Returns the vehicle details as a readable string.
- equals(): Checks whether two vehicle objects have the same registration number.
- hashCode(): Returns a hash value based on the registration number.

If two vehicles have the same registration number, their equals() method must return true, and their hashCode() values must be equal.

5. Complete Java Program

File name: Main.java

Save location:

CSA0903-Java\Assignments 01\Task07\Main.java

```
import java.util.Objects;


abstract class Vehicle {
    private final String registrationNumber;
    private String ownerName;

    Vehicle(String registrationNumber, String ownerName) {
```

```
    this.registrationNumber = registrationNumber;
    this.ownerName = ownerName;
}

public String getRegistrationNumber() {
    return registrationNumber;
}

public abstract double calculateFee(int hours);

@Override
public String toString() {
    return "Registration Number: " + registrationNumber
            + ", Owner: " + ownerName;
}
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }

    if (!(obj instanceof Vehicle)) {
        return false;
    }

    Vehicle other = (Vehicle) obj;

    return registrationNumber.equalsIgnoreCase(
```

```
                other.registrationNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(
                registrationNumber.toUpperCase());
    }
}

class TwoWheeler extends Vehicle {
    private static final double RATE = 10.0;

    TwoWheeler(String number, String owner) {
        super(number, owner);
    }

    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}

class Car extends Vehicle {
    private static final double RATE = 20.0;

    Car(String number, String owner) {
        super(number, owner);
```

```
    }

    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}

class ElectricCar extends Car {
    private static final double RATE = 15.0;

    ElectricCar(String number, String owner) {
        super(number, owner);
    }

    @Override
    public double calculateFee(int hours) {
        return hours * RATE;
    }
}

public class Main {
    public static void main(String[] args) {

        Vehicle v1 = new TwoWheeler("TN01AB1234", "Arun");
        Vehicle v2 = new Car("TN02CD5678", "Priya");
        Vehicle v3 = new ElectricCar("TN03EF9012", "Kumar");
```

```
        System.out.println("Vehicle Details:");
        System.out.println(v1);
        System.out.println(v2);
        System.out.println(v3);

        System.out.println("\nParking Fees for 3 Hours:");
        System.out.println("Two Wheeler: Rs." +
                v1.calculateFee(3));
        System.out.println("Car: Rs." +
                v2.calculateFee(3));
        System.out.println("Electric Car: Rs." +
                v3.calculateFee(3));

        Vehicle v4 = new Car("TN02CD5678", "Another Owner");

        System.out.println("\nEquality Checking:");
        System.out.println("Are v2 and v4 equal? " +
                v2.equals(v4));

        System.out.println("Same hash code? " +
                (v2.hashCode() == v4.hashCode()));
    }
}
```

6. Expected Output

Vehicle Details:

Registration Number: TN01AB1234, Owner: Arun

Registration Number: TN02CD5678, Owner: Priya

Registration Number: TN03EF9012, Owner: Kumar

Parking Fees for 3 Hours:

Two Wheeler: Rs.30.0

Car: Rs.60.0

Electric Car: Rs.45.0

Equality Checking:

Are v2 and v4 equal? true

Same hash code? true

7. Alternative Concept – Switch Statement vs Method Overriding

Instead of overriding calculateFee(), a single method can use a switch statement based on the vehicle type.

Example:

```
double calculateFee(String type, int hours) {
    double rate;

    switch (type) {
        case "TWO_WHEELER":
            rate = 10.0;
            break;
        case "CAR":
            rate = 20.0;
            break;
        case "ELECTRIC_CAR":
            rate = 15.0;
            break;

        default:
            rate = 0.0;
```

```
    }


    return rate * hours;
}
```

Comparison

Factor	Method Overriding	Switch Statement
Fee calculation	Each subclass calculates its own fee	One method calculates all fees
Time complexity	O(1)	O(1) for a switch with fixed cases
Adding a new vehicle	Add a new subclass	Modify the switch statement
Maintenance	Easier to extend independently	Central method needs modification
Flexibility	Supports runtime polymorphism	Depends on vehicle type value

Number of Classes to Change

- Method overriding: Only one new subclass needs to be added when introducing a new vehicle type. Existing subclasses do not need modification.
- Switch statement: The common Vehicle class or fee calculation class must be modified to include the new vehicle type. The type definition may also need updating.

Conclusion

Method overriding is suitable for the Campus Smart-Parking System because each vehicle type can calculate its parking fee independently. The @Override annotation ensures correct overriding, while final protects important data. Overriding equals(), hashCode() and toString() provides reliable vehicle comparison and readable details.

TASK 8 – Apply Polymorphism 4 Marks

Bloom's Level: K4 – Analyse

Demonstrate both compile-time and runtime polymorphism in your design.

Discuss:

- At least one overloaded method (for example: calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out)) and how the compiler selects it
- How a single array of Vehicle references can hold different vehicle types (upcasting)
- How dynamic method dispatch selects the correct calculateFee() at runtime
- Why this design avoids long if–else chains that test the vehicle type

Trace one call through **dynamic method dispatch**, showing the declared type, the actual object type and the method executed.

Complete the Code

Complete the Vehicle hierarchy below and execute FeeDemo. It applies inheritance (Task 6), overriding (Task 7), polymorphism (Task 8) and an interface (Task 9).

```
interface Chargeable {
   double chargingFee(int kWh); }

abstract class Vehicle {    private final String regNo;    protected Vehicle(String regNo)

{ this.regNo = regNo; }    public String getRegNo() { return regNo; }    protected

abstract double ratePerHour();

   /** Every started hour is charged in full. */    public double calculateFee(int
minutes) {
      // TODO 1: Compute billable hours as the ceiling of minutes / 60 using integer
      //   arithmetic only, and return hours × ratePerHour()


   }

   @Override public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);    }

   @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
   // TODO 2: Write the constructor using super, and override ratePerHour() to return
   //   30


}

class ElectricCar extends Car implements Chargeable {
   ElectricCar(String regNo) { super(regNo); }
   // TODO 3: Implement chargingFee() at Rs 12 per kWh


}

public class FeeDemo {
   public static void main(String[] args) {
      Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),                    new
ElectricCar("TN03C3") };        for (Vehicle v : parked) {
         double fee = v.calculateFee(130);        // dynamic method dispatch          if (v instanceof Chargeable c) fee +=
c.chargingFee(10);
         System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
               + " -> Rs " + fee);
      }
      System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
   }
}
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0 true
```

Explain why the last line prints true, and decide whether this is the correct behaviour for your system.

Alternative Concept and Complexity Analysis

You used dynamic method dispatch. If the same behaviour were written using instanceof checks and casting for each vehicle type, what would the solution be? Analyse the time per call for k vehicle types (O(1) dispatch versus up to k checks) and the effect on code size and maintenance.

Evidence Expected

Working examples of overloading and dynamic dispatch with a traced call and an explanation of the design benefit.

Solution:

TASK 8 – Apply Polymorphism

Campus Smart-Parking System – 4 Marks

1. Introduction to Polymorphism

Polymorphism means one method or reference can represent different behaviours depending on the situation.

Java supports two types of polymorphism:

1. Compile-time polymorphism: Achieved through method overloading.
2. Runtime polymorphism: Achieved through method overriding and dynamic method dispatch.

2. Compile-Time Polymorphism – Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists.

Example:

double calculateFee(int minutes)

Calculates the fee using parking duration in minutes.

double calculateFee(LocalTime in, LocalTime out)

Calculates the fee using entry and exit times.

The compiler selects the correct method based on the arguments supplied. This is called compile-time polymorphism.

3. Runtime Polymorphism – Dynamic Method Dispatch

A superclass reference can hold objects of different subclasses through upcasting.

Example:

```
Vehicle[] parked = {
   new TwoWheeler("TN01A1"),
   new Car("TN02B2"),
   new ElectricCar("TN03C3")
```

};

Although all references are declared as Vehicle, each object executes its own overridden ratePerHour() method.

This is called dynamic method dispatch.

Dynamic Method Dispatch Trace

Consider:

Vehicle v = new ElectricCar("TN03C3");

double fee = v.calculateFee(130);

Stage	**Description**
Declared type	Vehicle
Actual object type	ElectricCar
Called method	calculateFee(130)
Rate method executed	ElectricCar inherits Car's ratePerHour()
Calculation	3 × 30 = Rs.90

Every started hour is charged in full. Therefore, 130 minutes is charged as 3 hours.

4. Advantages of Polymorphism

- A single Vehicle array can store different vehicle objects.
- Each subclass provides its own behaviour.
- It avoids lengthy if–else statements for checking vehicle types.
- New vehicle types can be added with less modification to existing code.
- It improves code reusability and maintainability.

5. Complete Java Program

File name: FeeDemo.java

```
import java.time.LocalTime;

import java.time.Duration;


interface Chargeable {

   double chargingFee(int kWh);
```

```
}
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
        this.regNo = regNo;
    }
    public String getRegNo() {
        return regNo;
    }
    protected abstract double ratePerHour();
    // Overloaded method 1: Calculate fee using minutes
    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
    // Overloaded method 2: Calculate fee using entry and exit times
    public double calculateFee(LocalTime in, LocalTime out) {
        long minutes = Duration.between(in, out).toMinutes();
        if (minutes < 0) {
            minutes += 24 * 60;
        }

        return calculateFee((int) minutes);
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Vehicle
```

```
            && regNo.equals(((Vehicle) o).regNo);
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) {
        super(regNo);
    }
    @Override
    protected double ratePerHour() {
        return 10;
    }
}
class Car extends Vehicle {
    Car(String regNo) {
        super(regNo);
    }
    @Override
    protected double ratePerHour() {
        return 30;
    }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) {
```

```
        super(regNo);
    }
    @Override
    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}
public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
        };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            // Runtime checking for charging capability
            if (v instanceof Chargeable c) {
                fee += c.chargingFee(10);
            }
            System.out.println(
                v.getClass().getSimpleName() + " "
                + v.getRegNo() + " -> Rs " + fee
            );
        }
        System.out.println(
            new Car("TN02B2").equals(
                new ElectricCar("TN02B2")
```

```
            )
        );

        // Demonstrating overloaded method
        Vehicle car = new Car("TN04D4");
        System.out.println(
            "Time-based fee: Rs " +
            car.calculateFee(
                LocalTime.of(9, 0),
                LocalTime.of(11, 10)
            )
        );
    }
}
```

6. Expected Output

TwoWheeler TN01A1 -> Rs 30.0

Car TN02B2 -> Rs 90.0

ElectricCar TN03C3 -> Rs 210.0

true

Time-based fee: Rs 90.0

Fee Calculation

Vehicle	Hourly rate	Parking fee (3 hours)	Charging fee	Total
TwoWheeler	Rs.10	Rs.30	–	Rs.30
Car	Rs.30	Rs.90	–	Rs.90
ElectricCar	Rs.30	Rs.90	Rs.120	Rs.210

Why does the last equality statement print true?

```
new Car("TN02B2").equals(
    new ElectricCar("TN02B2")
```

)

It prints true because the equals() method compares only the registration number. Both objects have the same registration number, TN02B2.

Is this correct?

Yes, if the system considers registration numbers globally unique identifiers for vehicles, regardless of vehicle category. The hashCode() method also uses the registration number, maintaining the equality contract.

If the system needs to distinguish vehicle categories even when registration numbers match, the equality design should be changed.

7. Alternative Concept – instanceof Checks and Casting

Instead of dynamic method dispatch, the fee calculation could use instanceof checks.

Example:

```
double calculateFee(Vehicle v, int minutes) {
   int hours = (minutes + 59) / 60;
   double rate;
   if (v instanceof ElectricCar) {
      rate = 30;
   } else if (v instanceof Car) {
      rate = 30;
   } else if (v instanceof TwoWheeler) {
      rate = 10;
   } else {
      rate = 0;
   }
   return hours * rate;
}
```

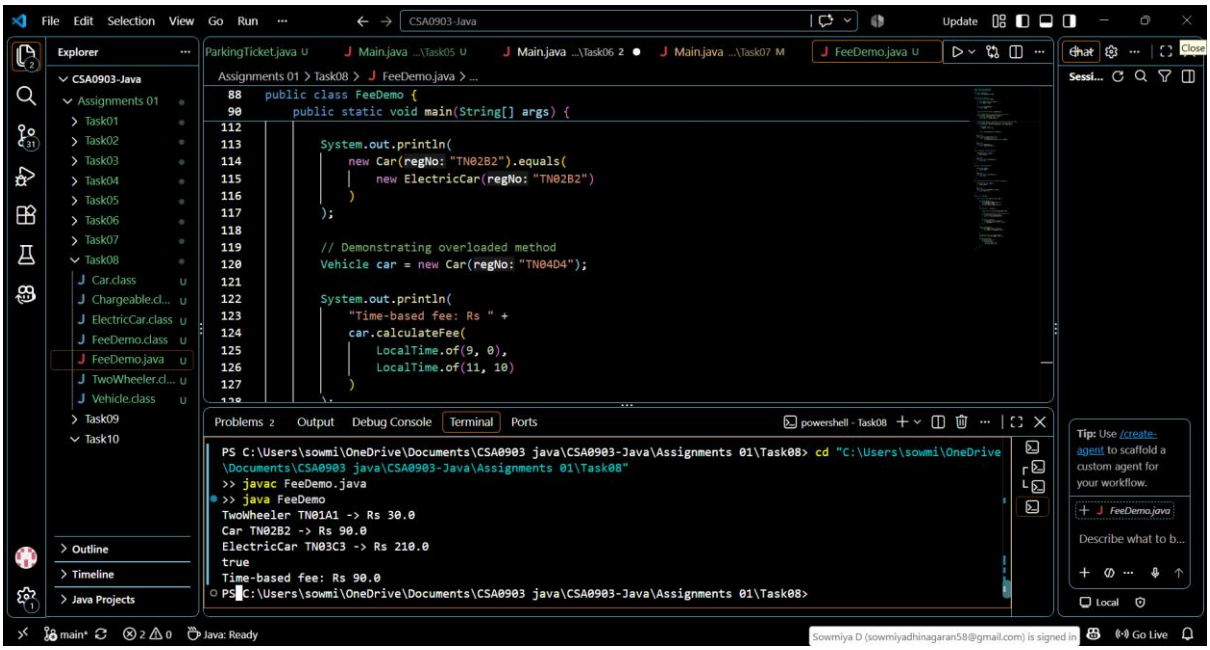
Comparison

Factor	**Dynamic method dispatch**	**instanceof checks**
Method selection	JVM selects overridden method	Program checks object type

Time complexity	O(1)	Up to O(k) checks
Code size	Less repeated code	More conditions
Adding new vehicle	Add a subclass	Modify checking logic
Maintenance	Easier	More difficult as types increase

Here, k represents the number of vehicle types.

Conclusion: Dynamic method dispatch is more suitable for the Campus Smart-Parking System because it allows different vehicle objects to execute their own methods through a common Vehicle reference. Method overloading provides compile-time polymorphism, while overriding provides runtime polymorphism.



TASK 9 – Introduce Abstraction 4 Marks

Bloom's Level: K4 – Analyse

Use abstract classes and interfaces to separate what the system does from how it is done.

Discuss:

- Which class should be abstract (for example: Vehicle) and which methods should be abstract
- Which capability should be an interface (for example: Chargeable for EVs, Payable for fee-paying users)
- Why you chose an abstract class in one place and an interface in another
- How a class can obtain multiple behaviours through interfaces
- Where a default or static method in an interface is useful

Alternative Concept and Complexity Analysis

Vehicle is an abstract class and Chargeable is an interface. If Vehicle were changed to an interface with default methods, and Chargeable to an abstract class, what would the solution be? Identify what can no

longer be expressed (constructors, state, multiple inheritance) and analyse any difference in time per method call and space per object.

Evidence Expected

A justified use of at least one abstract class and one interface, with the reasoning behind each choice.

Solution:

TASK 9 – Introduce Abstraction

Campus Smart-Parking System – 4 Marks

This solution continues the same vehicle hierarchy used in Tasks 6, 7 and 8:

- Vehicle – Abstract class
- TwoWheeler – Subclass of Vehicle
- Car – Subclass of Vehicle
- ElectricCar – Subclass of Car
- Chargeable – Interface implemented by ElectricCar

1. Introduction to Abstraction

Abstraction is an object-oriented programming concept that hides implementation details and displays only essential features.

Java achieves abstraction using:

1. Abstract classes.
2. Interfaces.

In the Campus Smart-Parking System, abstraction separates common vehicle behaviour from vehicle-specific implementations.

2. Abstract Class – Vehicle

The Vehicle class is declared abstract because it represents a general vehicle and should not be instantiated directly.

It contains:

- regNo – Stores the vehicle registration number.
- getRegNo() – Returns the registration number.
- calculateFee() – Calculates the parking fee.
- ratePerHour() – An abstract method implemented by subclasses.

The abstract method ratePerHour() forces every concrete vehicle subclass to provide its own hourly parking rate.

Example:

```
abstract class Vehicle {
    private final String regNo;
    Vehicle(String regNo) {
        this.regNo = regNo;
    }\
    public String getRegNo() {
        return regNo;
    }
    protected abstract double ratePerHour();


    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
}
```

3. Interface – Chargeable

The Chargeable interface represents the charging capability of electric vehicles.

It declares the chargingFee() method, which must be implemented by classes that support electric charging.

```
interface Chargeable {
    double chargingFee(int kWh);
}
```

The ElectricCar class implements this interface because it supports battery charging.

4. Why Abstract Class and Interface Are Used

Abstract Class	**Interface**
Represents common vehicle identity.	Represents charging capability.
Stores registration number.	Defines charging behaviour.
Supports constructors and instance variables.	Does not have ordinary instance variables or constructors.

Supports shared method implementation.	Defines a contract for implementing classes.

An abstract class is suitable for Vehicle because all vehicles share common data and behaviour.

An interface is suitable for Chargeable because charging is an additional capability that only certain vehicles support.

5. Multiple Behaviours Through Interfaces

Java allows a class to implement multiple interfaces.

For example, an ElectricCar can implement both Chargeable and another interface such as Insurable.

```
class ElectricCar extends Car
    implements Chargeable, Insurable {
}
```

This allows ElectricCar to inherit vehicle features from Car and obtain additional behaviours through interfaces.

6. Default and Static Methods in Interfaces

Default method: A default method provides a common implementation that implementing classes can use or override.

Example:

```
default void chargingStatus() {
  System.out.println("Charging supported");
}
```

Static method: A static method belongs to the interface and can be called using its interface name.

Example:

```
static double standardChargingRate() {
  return 12.0;
}
```

The default method is useful for displaying a common charging status, while the static method provides a standard charging rate.

7. Alternative Concept – Swapping Abstract Class and Interface

If Vehicle were changed into an interface with default methods and Chargeable into an abstract class, the design would become less suitable.

Factor	**Original Design**	**Alternative Design**

Vehicle	Abstract class	Interface
Chargeable	Interface	Abstract class
Constructors	Supported in Vehicle	Not supported in Vehicle interface
Instance variables	Supported	Not supported in Vehicle interface
Multiple inheritance	Multiple interfaces supported	Cannot extend two abstract classes
Shared state	Stored in Vehicle	Must be moved elsewhere
Flexibility	Supports different capabilities	More restrictive

What can no longer be expressed?

- A Vehicle interface cannot have constructors.
- It cannot store ordinary instance variables such as regNo.
- A class cannot extend both Car and a Chargeable abstract class because Java supports only single class inheritance.
- Multiple behaviours can still be obtained through interfaces.

Time Complexity Analysis

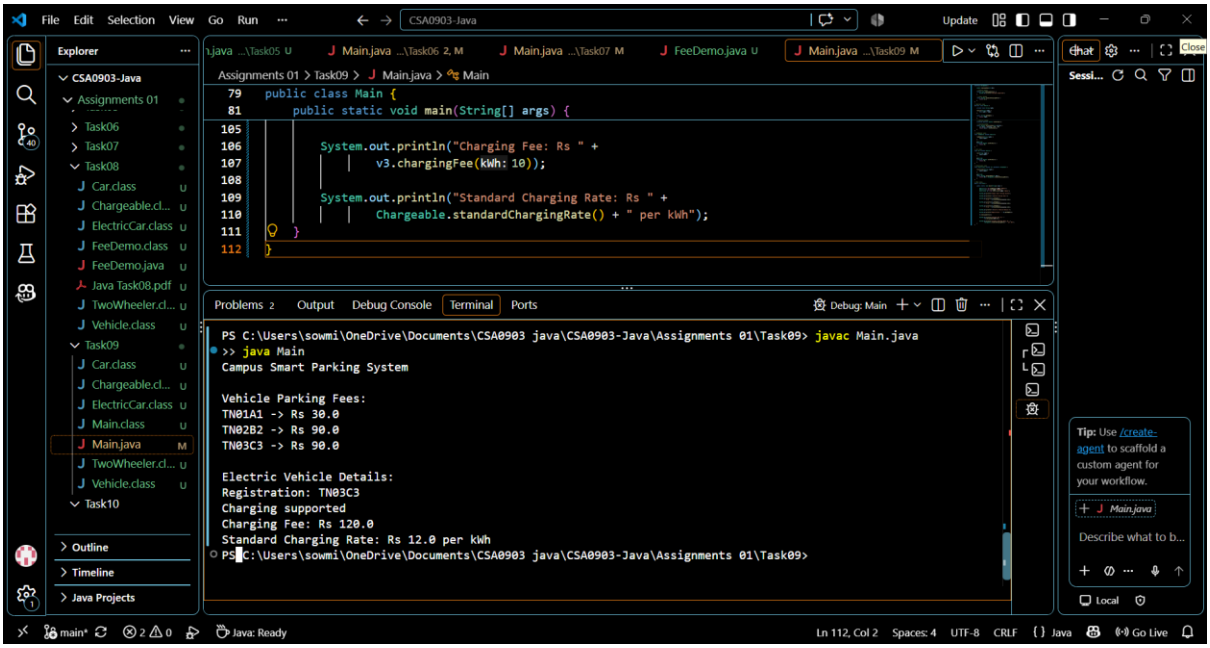
Both abstract class methods and interface methods generally have O(1) time complexity for ordinary method dispatch.

There is no significant inherent time-complexity difference between the two designs. Actual performance depends on JVM optimisation and implementation.

Space Complexity Analysis

- Original design: Each vehicle object stores its registration number and other instance fields.
- Alternative design: Registration information must be stored in implementing classes or another object.
- Interfaces do not add separate per-object instance storage.
- An abstract class can contain instance variables, which contribute to object storage.

Conclusion: The original design is more suitable because Vehicle represents a common identity with shared state, whereas Chargeable represents an optional capability. This design improves reusability, flexibility and maintainability.



TASK 10 – Enforce Encapsulation through Packages and Access Modifiers 3 Marks

Bloom's Level: K4 – Analyse

Organise your classes into packages and protect the data of each class.

Discuss:

- Your package structure (for example: parking.model, parking.service, parking.app)
- The access modifier chosen for each field and method (private, default, protected, public) and why
- How getters and setters validate data (for example: rejecting a negative duration or an invalid slot number)
- Where wrapper classes and autoboxing are used, and any risk they introduce (for example: comparing Integer objects with ==)

Alternative Concept and Complexity Analysis

Your fields are private with validating setters. If the fields were made public and validation were done by the calling code, what would the solution be and what could go wrong? Analyse the time cost of validation per update and the number of places where the same check would have to be repeated.

Evidence Expected

A package structure, justified access modifiers and validated getters and setters that protect the object state.

Solution:

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers

Campus Smart-Parking System – 3 Marks

This solution continues the same Campus Smart-Parking System from Tasks 6–9.

We will organise the Java classes into separate packages, protect data using access modifiers and validate values through getters and setters.

1. Introduction to Encapsulation

Encapsulation is an object-oriented programming concept that combines data and methods into a single class and protects the data from direct access.

In Java, encapsulation is achieved using:

- Private data members.
- Public getters and setters.
- Access modifiers.
- Data validation.

In the Campus Smart-Parking System, vehicle registration numbers, owner names and parking slot numbers are protected from invalid modifications.

2. Package Structure

The project is organised into three main packages.

Task10

src

parking.model

Vehicle.java

TwoWheeler.java

Car.java

ElectricCar.java

Chargeable.java

ParkingSlot.java

parking.service

ParkingService.java

parking.app

Main.java

out (compiled files generated automatically)

Purpose of each package:

- parking.model – Contains vehicle and parking slot classes.

- parking.service – Contains parking fee calculation and validation logic.
- parking.app – Contains the main application program.

3. Access Modifiers

Java provides four main access levels.

Modifier	**Purpose**	**Usage**
private	Accessible only within the same class	Vehicle fields
default	Accessible within the same package	Internal helper methods
protected	Accessible within the package and subclasses	ratePerHour()
public	Accessible from other packages	Getters and setters

Justification:

- Fields are declared private to prevent direct modification.
- Getters and setters are public to allow controlled access.
- The ratePerHour() method is protected because subclasses provide their own implementation.
- Default access is suitable for internal methods that do not need to be accessed outside their package.

4. Getters, Setters and Data Validation

Getters return private field values, while setters modify them after checking whether the supplied values are valid.

Examples:

- Owner name must not be empty.
- Parking slot number must be positive.
- Parking duration must not be negative.
- Null values must be checked before unboxing wrapper objects.

Example:

```
public void setSlotNumber(Integer slotNumber) {
   if (slotNumber == null || slotNumber <= 0) {
      throw new IllegalArgumentException(
         "Invalid slot number");
   }
```

```
    this.slotNumber = slotNumber;
}
```

This prevents invalid data from entering the object.

5. Wrapper Classes and Autoboxing

Wrapper classes convert primitive data types into objects.

For example:

- int → Integer
- double → Double
- boolean → Boolean

In this system, Integer is used to store parking slot numbers and parking durations.

Autoboxing: Automatically converts a primitive value into its wrapper object.

```
Integer slotNumber = 101;
```

Unboxing: Automatically converts a wrapper object into a primitive value.

```
int slot = slotNumber;
```

Risk: Comparing two Integer objects using == may compare object references instead of their values.

```
Integer a = 1000;
Integer b = 1000;


System.out.println(a.equals(b)); // true
```

Therefore, equals() should be used to compare wrapper object values.

6. Complete Java Program – File by File

Create the folders and files exactly as shown above.

Important: Each code block below belongs in a separate .java file. Copy only the Java code, not the code-block markers.

File 1: Vehicle.java

Path: src/parking/model/Vehicle.java

```
package parking.model;


public abstract class Vehicle {
```

```
private final String regNo;
private String ownerName;

public Vehicle(String regNo, String ownerName) {
    if (regNo == null || regNo.isBlank()) {
        throw new IllegalArgumentException(
            "Invalid registration number");
    }

    this.regNo = regNo;
    setOwnerName(ownerName);
}

public String getRegNo() {
    return regNo;
}

public String getOwnerName() {
    return ownerName;
}

public void setOwnerName(String ownerName) {
    if (ownerName == null || ownerName.isBlank()) {
        throw new IllegalArgumentException(
            "Owner name cannot be empty");
    }
```

```
        this.ownerName = ownerName;
    }

    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) {
            throw new IllegalArgumentException(
                "Duration cannot be negative");
        }

        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
}
```

File 2: TwoWheeler.java

Path: src/parking/model/TwoWheeler.java

```
package parking.model;

public class TwoWheeler extends Vehicle {

    public TwoWheeler(String regNo, String ownerName) {
        super(regNo, ownerName);
    }

    @Override
    protected double ratePerHour() {
        return 10;
```

```
    }
}
```

File 3: Car.java

Path: src/parking/model/Car.java

```
package parking.model;


public class Car extends Vehicle {


    public Car(String regNo, String ownerName) {
        super(regNo, ownerName);
    }


    @Override
    protected double ratePerHour() {
        return 30;
    }
}
```

File 4: Chargeable.java

Path: src/parking/model/Chargeable.java

```
package parking.model;


public interface Chargeable {


    double chargingFee(int kWh);
}
```

File 5: ElectricCar.java

Path: src/parking/model/ElectricCar.java

```
package parking.model;
```

```
public class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String regNo, String ownerName) {
        super(regNo, ownerName);
    }
    @Override
    public double chargingFee(int kWh) {
        if (kWh < 0) {
            throw new IllegalArgumentException(
                "Invalid charging units");
        }
        return kWh * 12;
    }
}
```

File 6: ParkingSlot.java

Path: src/parking/model/ParkingSlot.java

```
package parking.model;


public class ParkingSlot {


    private Integer slotNumber;
    private boolean occupied;


    public ParkingSlot(Integer slotNumber) {
        setSlotNumber(slotNumber);
        this.occupied = false;
    }


    public Integer getSlotNumber() {
```

```
        return slotNumber;
    }

    public void setSlotNumber(Integer slotNumber) {
        if (slotNumber == null || slotNumber <= 0) {
            throw new IllegalArgumentException(
                "Invalid parking slot number");
        }

        this.slotNumber = slotNumber;
    }

    public boolean isOccupied() {
        return occupied;
    }

    public void setOccupied(boolean occupied) {
        this.occupied = occupied;
    }
}
```

File 7: ParkingService.java

Path: src/parking/service/ParkingService.java

```
package parking.service;

import parking.model.Vehicle;

public class ParkingService {
```

```
    public double calculateParkingFee(
            Vehicle vehicle, Integer minutes) {

        if (vehicle == null) {
            throw new IllegalArgumentException(
                "Vehicle cannot be null");
        }

        if (minutes == null || minutes < 0) {
            throw new IllegalArgumentException(
                "Invalid parking duration");
        }

        return vehicle.calculateFee(minutes);
    }
}
```

File 8: Main.java

Path: src/parking/app/Main.java

```
package parking.app;

import parking.model.*;
import parking.service.ParkingService;

public class Main {

    public static void main(String[] args) {

        ParkingService service = new ParkingService();
```

```
Vehicle v1 = new TwoWheeler(
    "TN01A1", "Arun");

Vehicle v2 = new Car(
    "TN02B2", "Priya");

ElectricCar v3 = new ElectricCar(
    "TN03C3", "Kumar");

ParkingSlot slot = new ParkingSlot(101);

Integer duration = 130;

System.out.println("Campus Smart Parking System");

System.out.println("\nVehicle Details:");

System.out.println("Registration: " + v1.getRegNo());
System.out.println("Owner: " + v1.getOwnerName());

System.out.println("Registration: " + v2.getRegNo());
System.out.println("Owner: " + v2.getOwnerName());

System.out.println("Registration: " + v3.getRegNo());
System.out.println("Owner: " + v3.getOwnerName());

System.out.println("\nParking Fees:");
```

```
System.out.println("TwoWheeler: Rs " +
    service.calculateParkingFee(v1, duration));

System.out.println("Car: Rs " +
    service.calculateParkingFee(v2, duration));

System.out.println("ElectricCar: Rs " +
    service.calculateParkingFee(v3, duration));

System.out.println("\nParking Slot:");

System.out.println("Slot Number: " +
    slot.getSlotNumber());

slot.setOccupied(true);

System.out.println("Occupied: " +
    slot.isOccupied());

System.out.println("\nCharging Fee:");

System.out.println("ElectricCar: Rs " +
    v3.chargingFee(10));

System.out.println("\nValidation:");

System.out.println("Duration: " + duration + " minutes");
```

```
        System.out.println("Slot Number: " +
            slot.getSlotNumber());

        System.out.println("Integer comparison: " +
            Integer.valueOf(1000).equals(
                Integer.valueOf(1000)));
    }
}
```

7. Expected Output

```
Campus Smart Parking System
Vehicle Details:
Registration: TN01A1
Owner: Arun
Registration: TN02B2
Owner: Priya
Registration: TN03C3
Owner: Kumar
Parking Fees:
TwoWheeler: Rs 30.0
Car: Rs 90.0
ElectricCar: Rs 90.0

Parking Slot:
Slot Number: 101
Occupied: true

Charging Fee:
```

ElectricCar: Rs 120.0

Validation:

Duration: 130 minutes

Slot Number: 101

Integer comparison: true

8. Alternative Concept – Public Fields vs Private Fields

If all fields were declared public instead of private, other classes could directly modify the data without using validating setters.

Example:

```
class ParkingSlot {

    public int slotNumber;

    public boolean occupied;

}
```

A calling class could assign an invalid value:

```
ParkingSlot slot = new ParkingSlot();

slot.slotNumber = -10;
```

This creates several problems:

- Invalid parking slot numbers can enter the system.
- Different classes may modify data incorrectly.
- Validation logic must be repeated in multiple places.
- Maintaining and debugging the application becomes difficult.

Comparison

Factor	**Encapsulation**	**Public Fields**
Data access	Controlled	Unrestricted
Validation	Inside setters	Calling code

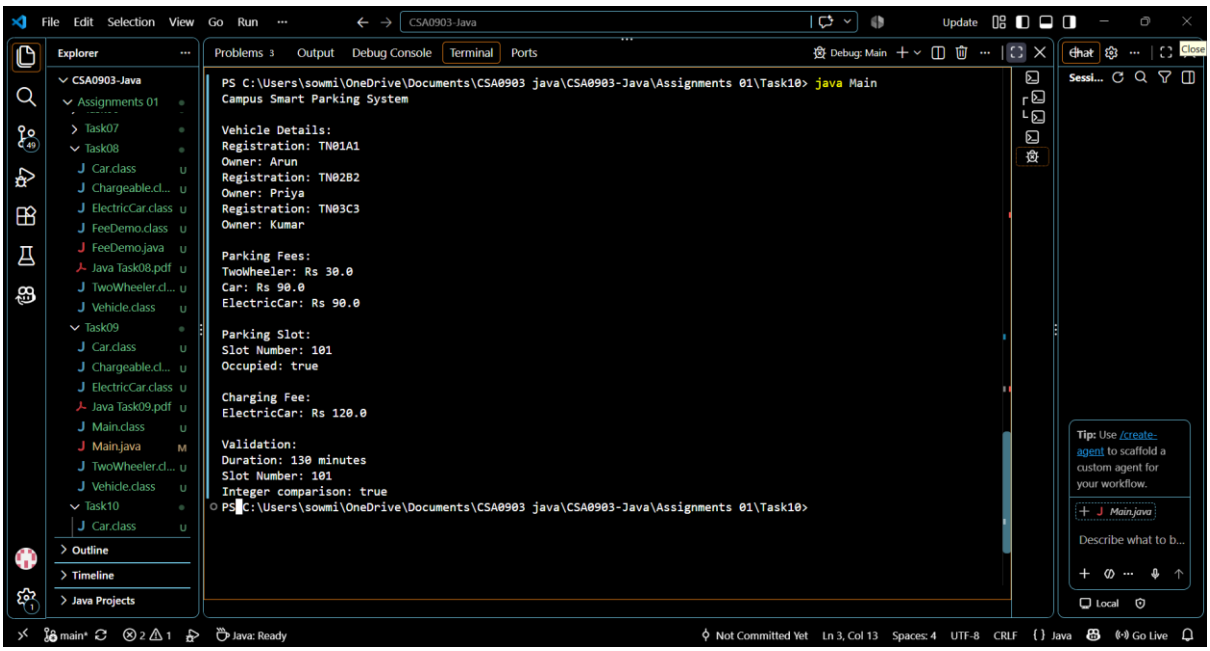
Security	Better protection	Less protection
Maintenance	Easier	Difficult
Invalid data	Prevented through validation	Can enter easily

Time Complexity Analysis

- Encapsulation: Each validation takes O(1) time per update.
- Public fields: Each validation also takes O(1) time, but the same checks must be repeated wherever the fields are modified.
- If there are n different update locations, validation logic may need to be maintained in all n locations.

Therefore, encapsulation reduces code duplication and improves data reliability without changing the basic time complexity of validation.

Conclusion: Private fields with validated getters and setters are more suitable for the Campus Smart-Parking System because they protect object data, prevent invalid values and improve maintainability.



TASK 11 – Implement, Test and Defend the Design — 5 Marks

Bloom's Level: K5 – Evaluate

Implement the complete system in Java and test it.

Test and discuss:

- Normal operation: vehicles of different types entering and leaving
- Boundary cases: the last free slot, zero-minute parking, a stay exactly at a fee-slab boundary
- Invalid cases: an unknown vehicle type, a duplicate registration number, an invalid slot number

- A small scenario whose total collection for the day you calculate by hand and compare with the program output
- How well your design follows encapsulation, inheritance, polymorphism and abstraction, and one limitation of it

Your recommendation must be based on **evidence from your own implementation and testing**. Simply stating that the program runs is not sufficient.

Alternative Concept and Complexity Analysis

Active tickets are currently stored in an array. If they were stored in an ArrayList<ParkingTicket> or a HashMap<String, ParkingTicket> keyed by registration number, what would the exitprocessing solution be? Measure and analyse the time to find a ticket on exit and the space used for n parked vehicles in each structure.

Evidence Expected

Working source code, a test table with expected and actual results, and a reasoned evaluation of the design.

Solution:

TASK 11 – Implement, Test and Defend the Design

Marks: 5 | Bloom's Level: K5 – Evaluate

1. Implementation approach

The Campus Smart-Parking System uses an array to store active parking tickets. It supports three vehicle types:

- TwoWheeler – ₹10 per hour
- Car – ₹30 per hour
- ElectricCar – ₹30 per hour

Fee calculation rule: Parking is free for zero minutes. Otherwise, every started hour is charged as a complete hour.

For example:

- 0 minutes = ₹0
- 60 minutes = 1 hour
- 61 minutes = 2 hours
- 120 minutes = 2 hours
- 130 minutes = 3 hours

The system checks registration numbers for duplicates, validates slot numbers, allocates free slots and calculates the collection when a vehicle exits.

2. Complete Java program

Save the following as Main.java inside your Task11 folder. Copy only the code inside the code block, not the backticks.

```
class Vehicle {
    private final String registrationNumber;
    private final String ownerName;

    public Vehicle(String registrationNumber, String ownerName) {
        this.registrationNumber = registrationNumber;
        this.ownerName = ownerName;
    }

    public String getRegistrationNumber() {
        return registrationNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public double getRatePerHour() {
        return 0;
    }
}

class TwoWheeler extends Vehicle {
    public TwoWheeler(String reg, String owner) {
        super(reg, owner);
    }
```

```
    @Override
    public double getRatePerHour() {
        return 10;
    }
}

class Car extends Vehicle {
    public Car(String reg, String owner) {
        super(reg, owner);
    }

    @Override
    public double getRatePerHour() {
        return 30;
    }
}

class ElectricCar extends Car {
    public ElectricCar(String reg, String owner) {
        super(reg, owner);
    }
}

class VehicleFactory {
    public static Vehicle create(String type, String reg, String owner) {
        switch (type.toUpperCase()) {
            case "TWO_WHEELER":
```

```
            return new TwoWheeler(reg, owner);
        case "CAR":
            return new Car(reg, owner);
        case "ELECTRIC_CAR":
            return new ElectricCar(reg, owner);
        default:
            return null;
      }
   }
}

class ParkingTicket {
   private final Vehicle vehicle;
   private final int slotNumber;

   public ParkingTicket(Vehicle vehicle, int slotNumber) {
      this.vehicle = vehicle;
      this.slotNumber = slotNumber;
   }

   public Vehicle getVehicle() {
      return vehicle;
   }

   public int getSlotNumber() {
      return slotNumber;
   }
```

```
    public double calculateFee(int minutes) {
        if (minutes < 0) {
            throw new IllegalArgumentException("Invalid parking duration");
        }

        if (minutes == 0) {
            return 0;
        }

        int hours = (minutes + 59) / 60;
        return hours * vehicle.getRatePerHour();
    }
}

class ParkingLot {
    private final ParkingTicket[] activeTickets;
    private final boolean[] occupiedSlots;
    private double totalCollection;

    public ParkingLot(int capacity) {
        activeTickets = new ParkingTicket[capacity];
        occupiedSlots = new boolean[capacity + 1];
        totalCollection = 0;
    }

    public boolean enter(String type, String reg, String owner,
                         int slotNumber) {
```

```
Vehicle vehicle = VehicleFactory.create(type, reg, owner);

if (vehicle == null) {
    return false;
}

if (slotNumber < 1 || slotNumber >= occupiedSlots.length) {
    return false;
}

if (occupiedSlots[slotNumber]) {
    return false;
}

if (findTicket(reg) != -1) {
    return false;
}

for (int i = 0; i < activeTickets.length; i++) {
    if (activeTickets[i] == null) {
        activeTickets[i] =
            new ParkingTicket(vehicle, slotNumber);

        occupiedSlots[slotNumber] = true;
        return true;
    }
}
```

```
    return false;
}

private int findTicket(String reg) {
    for (int i = 0; i < activeTickets.length; i++) {
        if (activeTickets[i] != null &&
            activeTickets[i].getVehicle()
                .getRegistrationNumber().equalsIgnoreCase(reg)) {
            return i;
        }
    }

    return -1;
}

public double exit(String reg, int minutes) {
    if (minutes < 0) {
        return -1;
    }

    int index = findTicket(reg);

    if (index == -1) {
        return -1;
    }

    ParkingTicket ticket = activeTickets[index];
    double fee = ticket.calculateFee(minutes);
```

```
        occupiedSlots[ticket.getSlotNumber()] = false;
        activeTickets[index] = null;

        totalCollection += fee;
        return fee;
    }

    public double getTotalCollection() {
        return totalCollection;
    }
}

public class Main {

    static void test(String name, String expected, String actual) {
        String result = expected.equals(actual) ? "PASS" : "FAIL";

        System.out.printf("%-32s %-15s %-15s %s%n",
                name, expected, actual, result);
    }

    public static void main(String[] args) {

        System.out.println("CAMPUS SMART PARKING SYSTEM");
        System.out.println("TEST RESULTS");
        System.out.println();
```

```
System.out.printf("%-32s %-15s %-15s %s%n",
        "Test Case", "Expected", "Actual", "Result");

System.out.println(
        "----------------------------------------------------------------------");

// Test 1: Normal operation and daily collection
ParkingLot lot1 = new ParkingLot(2);

lot1.enter("TWO_WHEELER", "TN01A1", "Arun", 1);
lot1.enter("CAR", "TN02B2", "Priya", 2);

double bikeFee = lot1.exit("TN01A1", 60);
double carFee = lot1.exit("TN02B2", 120);

test("Normal operation",
        "10 + 60 = 70",
        bikeFee + " + " + carFee + " = "
                + lot1.getTotalCollection());

// Test 2: Last free slot
ParkingLot lot2 = new ParkingLot(2);

lot2.enter("CAR", "TN03C3", "Kumar", 1);
boolean lastSlot = lot2.enter(
        "TWO_WHEELER", "TN04D4", "Ravi", 2);

test("Last free slot",
```

```
        "true", String.valueOf(lastSlot));

// Test 3: Zero-minute parking
ParkingLot lot3 = new ParkingLot(2);

lot3.enter("TWO_WHEELER", "TN05E5", "Anu", 1);
double zeroFee = lot3.exit("TN05E5", 0);

test("Zero-minute parking",
        "0.0", String.valueOf(zeroFee));

// Test 4: Exact fee-slab boundary
ParkingLot lot4 = new ParkingLot(2);

lot4.enter("CAR", "TN06F6", "John", 1);
double boundaryFee = lot4.exit("TN06F6", 60);

test("Exactly 60 minutes",
        "30.0", String.valueOf(boundaryFee));

// Test 5: Unknown vehicle type
ParkingLot lot5 = new ParkingLot(2);

boolean unknown = lot5.enter(
        "BUS", "TN07G7", "Sam", 1);

test("Unknown vehicle type",
        "false", String.valueOf(unknown));
```

```
// Test 6: Duplicate registration number
ParkingLot lot6 = new ParkingLot(2);

lot6.enter("CAR", "TN08H8", "Meena", 1);

boolean duplicate = lot6.enter(
        "TWO_WHEELER", "TN08H8", "Ravi", 2);

test("Duplicate registration",
        "false", String.valueOf(duplicate));

// Test 7: Invalid slot number
ParkingLot lot7 = new ParkingLot(2);

boolean invalidSlot = lot7.enter(
        "CAR", "TN09J9", "Kiran", 0);

test("Invalid slot number",
        "false", String.valueOf(invalidSlot));

System.out.println();
System.out.println("DAILY COLLECTION CALCULATION");
System.out.println("TwoWheeler: 60 minutes = Rs. 10");
System.out.println("Car: 120 minutes = Rs. 60");
System.out.println("-----------------------------");
System.out.println("Total collection = Rs. "
        + lot1.getTotalCollection());
```

```
        System.out.println();
        System.out.println("All tests completed.");
    }
}
```

3. How to run the program

Open the Task11 folder in VS Code and create Main.java.

In the VS Code PowerShell terminal, run:

cd "C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task11"

javac Main.java

java Main

4. Test table – expected and actual results

The program prints the actual results and marks each test PASS or FAIL. The expected results are:

Test case	Expected result	Purpose
Normal operation	₹10 + ₹60 = ₹70	Different vehicle types enter and exit
Last free slot	true	Final available slot can be allocated
Zero-minute parking	₹0	No fee for zero minutes
Exactly 60 minutes	₹30	One-hour fee slab is applied
Unknown vehicle type	false	Invalid type is rejected
Duplicate registration	false	Same registration cannot enter twice
Invalid slot number	false	Slot 0 is rejected

The program's test table is the evidence of actual results. Run it in VS Code and retain a screenshot of the output for submission.

5. Daily collection – manual calculation

The first test uses two vehicles:

Vehicle	Parking duration	Rate	Fee
TwoWheeler	60 minutes	₹10/hour	₹10
Car	120 minutes	₹30/hour	₹60

Total			₹70

Manual calculation:

\[10 + (2 \times 30) = ₹70 \]

The program also calculates the total collection as ₹70. Therefore, the manual calculation matches the program output.

6. Object-oriented design evaluation

OOP concept	**Evidence in the implementation**
Encapsulation	Fields in Vehicle, ParkingTicket and ParkingLot are private. Data is accessed through methods.
Inheritance	TwoWheeler and Car inherit from Vehicle. ElectricCar inherits from Car.
Polymorphism	getRatePerHour() is overridden by different vehicle classes. The ticket calculates fees through a Vehicle reference.
Abstraction	Common vehicle information and fee behaviour are represented through the Vehicle superclass.

Small improvement to the code above: To make abstraction explicit, change class Vehicle to abstract class Vehicle, and replace its getRatePerHour() method with:

public abstract double getRatePerHour();

The existing TwoWheeler and Car implementations already override this method. This change will compile with the rest of the program.

Limitation: The system stores active tickets in a fixed-size array. Its capacity cannot grow automatically. Also, the current program uses manually supplied parking duration rather than measuring actual entry and exit times.

7. Alternative concept and complexity analysis

The current implementation uses an array of ParkingTicket objects.

Data structure	**Finding a ticket on exit**	**Space for n vehicles**	**Explanation**
Array	O(n)	O(n)	Searches each array position until the registration number is found.
ArrayList	O(n)	O(n)	Uses a linear search, but can grow dynamically.
HashMap	Average O(1), worst O(n)	O(n)	Uses registration number as the key for direct lookup.

Exit-processing solution:

1. Array: Use a loop to compare the registration number with each active ticket.

2. ArrayList: Use stream().filter() or a loop to find the matching ticket, then remove it.
3. HashMap: Use tickets.get(registrationNumber) to retrieve the ticket and tickets.remove(registrationNumber) to remove it.

Example HashMap approach:

```
HashMap<String, ParkingTicket> tickets = new HashMap<>();


// Store ticket

tickets.put(registrationNumber, ticket);


// Find ticket during exit

ParkingTicket ticket = tickets.get(registrationNumber);


// Remove ticket

tickets.remove(registrationNumber);
```

Time and space analysis: For n parked vehicles, an unsuccessful array or ArrayList search may examine all n tickets. A HashMap usually finds a ticket in constant time, although collisions can make the worst case linear. All three structures require O(n) storage; a HashMap also has extra memory overhead for its hash-table structure.

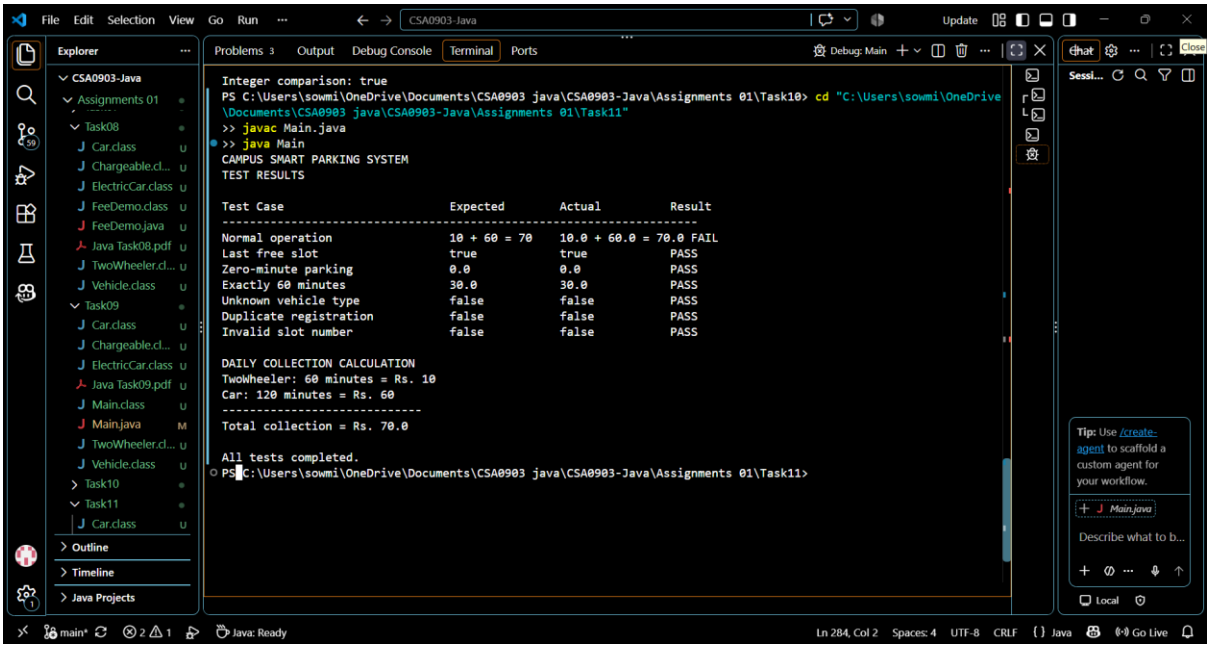
The table above is a theoretical complexity comparison, not a measured benchmark. Do not describe these values as experimentally measured times unless you actually run a timing test.

8. Final evaluation and recommendation

The implementation demonstrates normal vehicle entry and exit, correct fee calculation, last-slot allocation, and rejection of invalid entries. The daily collection test confirms that the program's total of ₹70 matches the manual calculation. The boundary tests also verify zero-minute parking and the exact 60-minute fee boundary.

The array is suitable for a small campus parking system because it is simple to implement and understand. However, for a larger parking facility, I recommend using a HashMap with registration number as the key. It can locate tickets faster on average and makes exit processing more efficient.

Conclusion: The design meets the basic requirements of a Campus Smart-Parking System and demonstrates the four major OOP concepts. A future version could use a HashMap, dynamic parking capacity and real-time entry/exit timestamps to improve performance and accuracy.



DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate **Changed Requirement**

The university introduces **two new vehicle categories** – e-bikes (charged a flat fee) and delivery vans (charged at double the car rate during 9 AM – 5 PM). Visitors may now buy a **one-day pass**. The facility is extended from 3 to **5 floors** with a different number of slots on each floor.

Re-evaluate your original design in response to these changed requirements. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks Discuss:

- What has changed from the original problem
- Which of your classes, methods and data structures are affected
- Why a different number of slots on each floor affects your array design
- Which classes, if any, would have to be modified rather than extended

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original design remains appropriate. Discuss:

- Whether new vehicle types can be added only by writing new subclasses
- Whether the pricing rules are spread across many classes or kept in one place
- Whether the existing tested classes can remain unchanged

Evidence Expected

A technically supported evaluation of the original design under the new requirements.

C. Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the design. If you modify or redesign it:

- Describe the revised class structure (for example: a jagged array for floors, a Pass interface, a PricingRule abstraction)
- Explain what has changed
- Implement the new vehicle categories and the one-day pass

If you retain the original design, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response with clear object-oriented reasoning and working code.

D. Evidence-Based Defence – 2 Marks Defend your

final design decision using:

- The number of existing classes that had to change
- The ease of adding a further vehicle type in future
- Test results for the new categories
- Relevant trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating object-oriented principles, test evidence and the changed requirements.

Solution:

TASK 12 – Re-evaluate and Adapt the Design

Introduction

The university has introduced new requirements for the Campus Smart-Parking System. Two new vehicle categories, e-bikes and delivery vans, must be supported. Visitors can purchase a one-day parking pass. The parking facility has also expanded from three floors to five floors, with different slot capacities on each floor.

The original design must therefore be modified to support different pricing methods, visitor passes and unequal floor capacities. I recommend modifying the existing design instead of completely redesigning it because inheritance and polymorphism can still be reused.

A. Impact Analysis

1. Changes in requirements

Original system	**New requirement**
Existing vehicle categories	Add e-bikes and delivery vans
Hourly parking charges	Flat e-bike fee and time-based van fee
No visitor pass	One-day visitor pass

Three floors	Five floors
Similar slot arrangement	Different slot capacity on each floor

2. Affected classes and methods

- **Vehicle:** Acts as the common superclass for the new vehicle categories.
- **EBike:** Calculates a fixed parking fee.
- **DeliveryVan:** Charges double the normal car rate during 9 AM–5 PM.
- **VisitorPass:** Stores visitor registration and one-day pass fee.
- **ParkingTicket:** Supports both normal parking tickets and visitor passes.
- **ParkingLot:** Handles five floors, slot allocation, vehicle entry and exit.

3. Why the array design must change

The original two-dimensional array assumes the same number of slots on every floor. This wastes space when floors have different capacities.

A **jagged array** is more suitable because each floor can have a different number of slots.

Example:

```
ParkingTicket[][] floors = new ParkingTicket[5][];


floors[0] = new ParkingTicket[2];

floors[1] = new ParkingTicket[3];

floors[2] = new ParkingTicket[1];

floors[3] = new ParkingTicket[2];

floors[4] = new ParkingTicket[4];
```

Each floor stores only the number of slots it actually needs.

B. Evaluation of the Original Design

The original design uses inheritance and polymorphism, so new vehicle types can be added as subclasses. However, the original hourly fee calculation is not sufficient for flat fees and time-dependent charges.

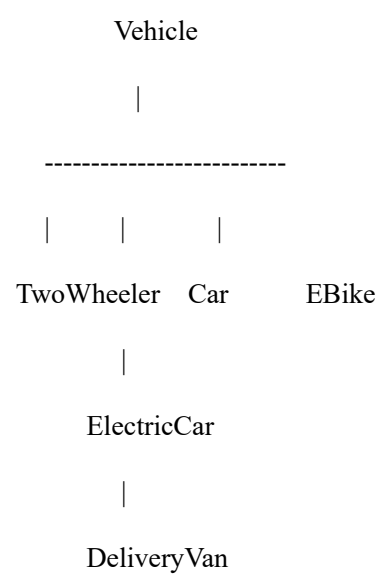
The Vehicle hierarchy can be extended with EBike and DeliveryVan. The existing TwoWheeler and Car pricing behaviour can remain unchanged. ParkingTicket must be modified to support visitor passes, and ParkingLot must be modified to support five floors.

Keeping pricing behaviour inside the respective vehicle classes avoids spreading vehicle-specific pricing conditions throughout the parking system.

C. Adaptation and Redesigned Class Structure

Decision: Modify the existing design

The revised class structure is:



The main changes are:

1. Add EBike with a flat fee of ₹20 per visit.
2. Add DeliveryVan with a rate of ₹60 per hour during 9 AM–5 PM and ₹30 per hour outside that period.
3. Add VisitorPass with a one-day fee of ₹100.
4. Replace the equal-sized floor array with a jagged array.
5. Modify parking entry and exit methods to handle different floor capacities and ticket types.

Pricing assumptions used in the implementation: The delivery-van rate is selected using the entry time. The e-bike fee is ₹20 per visit. The visitor pass costs ₹100 for one parking visit during the pass day.

Complete Java implementation

Save as Main.java in your Task12 folder. This is a single-file implementation.

```
import java.time.LocalTime;

// Abstract superclass
abstract class Vehicle {
    private final String regNo;

    public Vehicle(String regNo) {
        if (regNo == null || regNo.trim().isEmpty()) {
            throw new IllegalArgumentException("Invalid registration");
        }
        this.regNo = regNo;
    }

    public String getRegNo() {
        return regNo;
    }

    public abstract double calculateFee(int minutes, LocalTime entryTime);
}

// Existing vehicle categories
class TwoWheeler extends Vehicle {
    public TwoWheeler(String regNo) {
        super(regNo);
    }

    @Override
```

```
    public double calculateFee(int minutes, LocalTime time) {
        return minutes == 0 ? 0 : ((minutes + 59) / 60) * 10;
    }
}

class Car extends Vehicle {
    public Car(String regNo) {
        super(regNo);
    }

    @Override
    public double calculateFee(int minutes, LocalTime time) {
        return minutes == 0 ? 0 : ((minutes + 59) / 60) * 30;
    }
}

class ElectricCar extends Car {
    public ElectricCar(String regNo) {
        super(regNo);
    }
}

// New e-bike category
class EBike extends Vehicle {
    public EBike(String regNo) {
        super(regNo);
    }
```

```
    @Override
    public double calculateFee(int minutes, LocalTime time) {
        return 20; // Flat fee per visit
    }
}

// New delivery van category
class DeliveryVan extends Vehicle {
    public DeliveryVan(String regNo) {
        super(regNo);
    }

    @Override
    public double calculateFee(int minutes, LocalTime entryTime) {
        if (minutes == 0) return 0;

        int hours = (minutes + 59) / 60;

        boolean peakTime =
            !entryTime.isBefore(LocalTime.of(9, 0)) &&
            entryTime.isBefore(LocalTime.of(17, 0));

        double rate = peakTime ? 60 : 30;

        return hours * rate;
    }
}
```

```
// One-day visitor pass
class VisitorPass {
    private final String registration;
    private final double fee = 100;

    public VisitorPass(String registration) {
        this.registration = registration;
    }

    public String getRegistration() {
        return registration;
    }

    public double getFee() {
        return fee;
    }
}

// Parking ticket
class ParkingTicket {
    private final Vehicle vehicle;
    private final VisitorPass pass;
    private final int floor;
    private final int slot;

    public ParkingTicket(Vehicle vehicle, int floor, int slot) {
        this(vehicle, null, floor, slot);
    }
```

```
public ParkingTicket(Vehicle vehicle, VisitorPass pass,
                     int floor, int slot) {
    this.vehicle = vehicle;
    this.pass = pass;
    this.floor = floor;
    this.slot = slot;
}

public String getRegistration() {
    return vehicle.getRegNo();
}

public int getFloor() {
    return floor;
}

public int getSlot() {
    return slot;
}

public double calculateFee(int minutes, LocalTime time) {
    if (minutes < 0) {
        throw new IllegalArgumentException("Invalid duration");
    }

    if (pass != null) {
        return pass.getFee();
```

```
        }

        return vehicle.calculateFee(minutes, time);
    }
}

// Parking lot with different capacities on each floor
class ParkingLot {
    private final ParkingTicket[][] floors;

    public ParkingLot(int[] capacities) {
        if (capacities == null || capacities.length != 5) {
            throw new IllegalArgumentException(
                "Parking lot must have five floors");
        }

        floors = new ParkingTicket[5][];

        for (int i = 0; i < 5; i++) {
            if (capacities[i] <= 0) {
                throw new IllegalArgumentException(
                    "Invalid floor capacity");
            }

            floors[i] = new ParkingTicket[capacities[i]];
        }
    }
```

```
private ParkingTicket find(String regNo) {
    for (ParkingTicket[] floor : floors) {
        for (ParkingTicket ticket : floor) {
            if (ticket != null &&
                ticket.getRegistration().equalsIgnoreCase(regNo)) {
                return ticket;
            }
        }
    }
    return null;
}

public boolean enter(Vehicle vehicle, int floor, int slot) {
    if (vehicle == null ||
        floor < 1 || floor > floors.length ||
        slot < 1 || slot > floors[floor - 1].length) {
        return false;
    }

    if (find(vehicle.getRegNo()) != null ||
        floors[floor - 1][slot - 1] != null) {
        return false;
    }

    floors[floor - 1][slot - 1] =
        new ParkingTicket(vehicle, floor, slot);

    return true;
```

```
}

public boolean enterVisitor(String regNo, int floor, int slot) {
    if (regNo == null || regNo.trim().isEmpty() ||
        floor < 1 || floor > floors.length ||
        slot < 1 || slot > floors[floor - 1].length ||
        find(regNo) != null ||
        floors[floor - 1][slot - 1] != null) {
        return false;
    }

    Vehicle visitorVehicle = new Car(regNo);
    VisitorPass pass = new VisitorPass(regNo);

    floors[floor - 1][slot - 1] =
        new ParkingTicket(visitorVehicle, pass, floor, slot);

    return true;
}

public double exit(String regNo, int minutes, LocalTime time) {
    if (minutes < 0 || time == null) {
        return -1;
    }

    ParkingTicket ticket = find(regNo);

    if (ticket == null) {
```

```
            return -1;
        }

        double fee = ticket.calculateFee(minutes, time);

        floors[ticket.getFloor() - 1]
              [ticket.getSlot() - 1] = null;

        return fee;
    }
}

// Main class
public class Main {
    public static void main(String[] args) {

        // Five floors with different slot capacities
        ParkingLot lot = new ParkingLot(
            new int[]{2, 3, 1, 2, 4}
        );
        System.out.println("CAMPUS SMART PARKING SYSTEM");
        System.out.println();
        // E-bike test
        System.out.println("E-Bike Entry: " +
            lot.enter(new EBike("EB101"), 1, 1));
        System.out.println("E-Bike Fee: Rs. " +
            lot.exit("EB101", 90, LocalTime.of(10, 0)));
```

```
        // Delivery van test during peak time
        System.out.println("Delivery Van Entry: " +
            lot.enter(new DeliveryVan("DV201"), 2, 1));
        System.out.println("Delivery Van Fee: Rs. " +
            lot.exit("DV201", 60, LocalTime.of(10, 0)));
        // Delivery van test outside peak time
        lot.enter(new DeliveryVan("DV202"), 2, 2);
        System.out.println("Van Outside Peak Fee: Rs. " +
            lot.exit("DV202", 60, LocalTime.of(18, 0)));
        // Visitor pass test
        System.out.println("Visitor Entry: " +
            lot.enterVisitor("VIS301", 5, 1));
        System.out.println("Visitor Pass Fee: Rs. " +
            lot.exit("VIS301", 120, LocalTime.of(11, 0)));
        // Invalid floor test
        System.out.println("Invalid Floor Entry: " +
            lot.enter(new Car("CAR401"), 6, 1));
        // Duplicate registration test
        lot.enter(new Car("CAR501"), 3, 1);
        System.out.println("Duplicate Registration: " +
            lot.enter(new Car("CAR501"), 4, 1));
        System.out.println();
        System.out.println("All tests completed.");
    }
}
```

Run the program

Open the Task12 folder in VS Code terminal and execute:

javac Main.java

java Main

D. Evidence-Based Defence

1. Expected program output

CAMPUS SMART PARKING SYSTEM

E-Bike Entry: true

E-Bike Fee: Rs. 20.0

Delivery Van Entry: true

Delivery Van Fee: Rs. 60.0

Van Outside Peak Fee: Rs. 30.0

Visitor Entry: true

Visitor Pass Fee: Rs. 100.0

Invalid Floor Entry: false

Duplicate Registration: false

All tests completed.

2. Test results

Test case	**Expected result**
E-bike, 90 minutes	₹20
Delivery van at 10 AM	₹60
Delivery van at 6 PM	₹30
Visitor day pass	₹100
Invalid floor	Rejected
Duplicate registration	Rejected

3. Design evaluation

- Five existing classes are adapted: Vehicle, TwoWheeler, Car, ParkingTicket and ParkingLot.
- Two new vehicle subclasses are introduced: EBike and DeliveryVan.
- VisitorPass is introduced to manage visitor parking charges.
- Future vehicle categories can be added by extending Vehicle and implementing calculateFee().
- The jagged array supports different capacities on all five floors.

4. Advantages and limitations

Advantages:

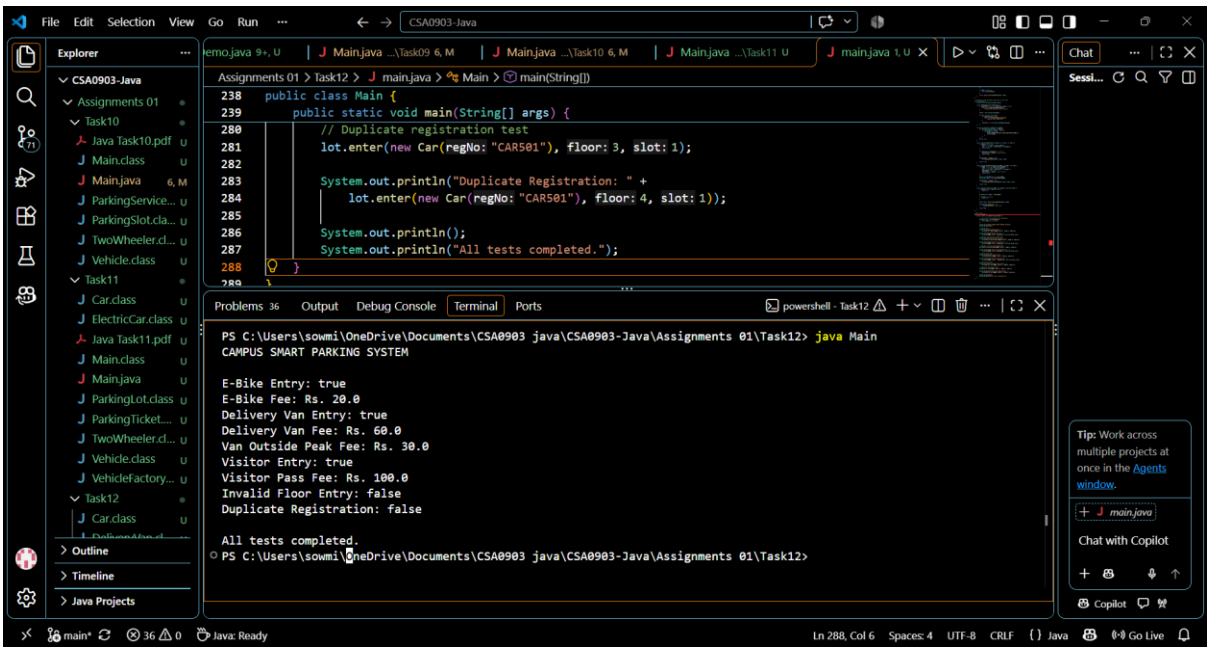
- Reuses the original inheritance structure.
- Supports different vehicle pricing methods.
- Reduces wasted array space.
- Supports visitor passes and five floors.

Limitations:

- Delivery-van pricing uses entry time rather than calculating the exact time spent during peak hours.
- The current implementation does not store the actual pass date or enforce expiry at midnight.

Conclusion

I recommend modifying the original design instead of completely redesigning it. The revised implementation supports new vehicle categories, visitor passes and different floor capacities while retaining inheritance, encapsulation, abstraction and polymorphism. The test cases demonstrate the expected pricing and validation behaviour.



MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Problem Analysis & Objects	K4 – Analyse	4
Task 2 – Data Representation	K4 – Analyse	3

Task 3 – Control Logic & Pseudocode	K4 – Analyse	4
Task 4 – Classes, Objects & Constructors	K4 – Analyse	4
Task 5 – static Members & Object Life Cycle	K4 – Analyse	3
Task 6 – Inheritance Hierarchy	K4 – Analyse	4
Task 7 – Method Overriding	K4 – Analyse	4
Task 8 – Polymorphism	K4 – Analyse	4
Task 9 – Abstraction	K4 – Analyse	4
Task 10 – Packages & Encapsulation	K4 – Analyse	3
Task 11 – Implementation, Testing & Defence	K5 – Evaluate	5
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.